

2Chance Web Application

Sustainable Software Engineering Project Report

Aldo Manco

23 January 2026

Contents

1	Introduzione e Baseline del Sistema	3
2	Panoramica dell'Architettura del Software	4
2.1	Layered Architecture Components Breakdown	4
2.1.1	The View Layer (Presentation and Interaction)	4
2.1.2	The Controller Layer (Input Coordination and Routing)	4
2.1.3	The Service Layer (Business Logic Orchestration)	5
2.1.4	The Data Access Layer (Persistence and DAO)	5
2.1.5	The Model Beans (Data Representation)	5
2.2	Dynamic Behavior: The Authentication Workflow	6
3	Sostenibilità Energetica del Back-End	7
3.1	Misurazione Dinamica dell'Energia tramite EnergiBridge e JMeter	7
3.1.1	Configurazione dell'Ambiente di Profilazione	7
3.1.2	Strategia di Monitoraggio e Workload Deterministico	7
3.1.3	Progettazione degli Scenari di Test JMeter	9
3.1.4	Definizione delle Soglie di Accettabilità (SLAs) e Dimensionamento dei Test	11
3.1.5	Automazione, Isolamento e Sincronizzazione dei Test (backend_energy_consumption_anal	
3.1.6	Analisi Energetica Post-Elaborazione ed Estrazione delle Metriche (energibridge_metrics	
3.2	Analisi Statica del Codice (Creedengo)	15
3.2.1	Configurazione di SonarQube con il Plugin Creedengo	16
3.2.2	Selezione delle Regole di Eco-Design	16
3.2.3	Risultati dell'Analisi Statica e Risoluzione dei Fault	17
3.3	Microbenchmarking Energetico (JMH)	17
3.3.1	Anti-Pattern di Sostenibilità Identificati	17
3.3.2	Microbenchmarking Energetico (JMH)	18
3.3.3	Strumento di Profilazione Energetica Co-Sincronizzato: Architettura e Teoria Operativa	18
3.3.4	Risultati Sperimentali dei Benchmark (Smoke Runs)	18
3.4	Piano di Refactoring ed Ottimizzazione	19
3.5	Validazione delle Ottimizzazioni	20
3.5.1	Validazione dell'Analisi Statica (Creedengo)	20
3.5.2	Risultati Sperimentali dei Benchmark Post-Refactoring	20
3.6	Confronto Energetico di Sistema Post-Refactoring (JMeter)	21
3.6.1	Correlazione delle Ottimizzazioni con i Consumi Energetici di Sistema	22
4	Valutazione e Ottimizzazione Ambientale del Livello Front-End	24
4.1	Analisi Ambientale del Baseline (GreenIT ed EcoIndex)	24
4.1.1	Metodologia di Valutazione: Contesto Reale (GreenIT-Analysis) vs Contesto Laboratorio (EcoIndex)	25

4.1.2	Dettaglio dell'Inefficienza: Ridimensionamento Immagini nel Browser . .	25
4.1.3	Dettaglio dell'Inefficienza: Download di Risorse Nascoste (Unnecessary Downloads)	26
4.1.4	Dettaglio dell'Inefficienza: Assenza di un Foglio di Stile di Stampa (Print Stylesheet)	26
4.1.5	Dettaglio dell'Inefficienza: Mancanza di Directive di Caching (Caching Headers)	26
4.1.6	Impatto Teorico ed Ecologico del Ridimensionamento Client-Side	27
4.2	Interventi di Refactoring e Ottimizzazione Ambientale	27
4.2.1	Ottimizzazione Fisica degli Asset e Generazione Miniature	27
4.2.2	Integrazione a Livello Backend (Java Bean)	28
4.2.3	Aggiornamento dei Livelli JSP e CSS	28
4.2.4	Appiattimento del DOM e Lazy Loading in showProdotti.jsp	28
4.2.5	Implementazione del Caricamento Differito (Lazy Loading) del Carosello	29
4.2.6	Integrazione del Foglio di Stile di Stampa per l'Eco-Progettazione	29
4.2.7	Ottimizzazione dei Tempi di Caching (Cache-Control ed Expires)	30
4.2.8	Compressione GZIP delle Risorse (CompressionFilter)	30
4.2.9	Risoluzione dell'Errore HTTP 404 per il Favicon (favicon.ico)	31
4.2.10	Risoluzione degli Errori HTTP 500 per la Directory Immagini (/img/) .	31
4.2.11	Eliminazione dei Cookie per le Risorse Statiche (Cookie-Free Assets) . .	32
4.2.12	Test di Correttezza e Integrità	32
4.2.13	Analisi della Best Practice "Use HTTP/2 instead of HTTP/1"	32
4.3	Bypass dell'Autenticazione nei Test Ambientali (Test Mode Session Injector) . .	34
4.3.1	Quantificazione dell'Ottimizzazione Globale	37
4.3.2	Nota Metodologica sulle Fluttuazioni del Punteggio EcoIndex	38
5	Monitoraggio Licenze Open Source	39
5.1	Analisi della Compliance e dei Rischi Legali (FOSSA)	39
5.1.1	Sostenibilità Legale ed Economica: Software Composition Analysis e Conformità delle Licenze	39
5.1.2	Monitoraggio delle Licenze Open Source tramite FOSSA	39
5.1.3	Esecuzione dell'Analisi FOSSA	40
5.1.4	Albero delle Dipendenze del Progetto 2Chance	41
5.1.5	Inventario Completo delle Licenze	42
5.1.6	Distribuzione delle Licenze	43
5.1.7	Valutazione del Rischio Copyleft	43
5.1.8	Sintesi del Rischio e Raccomandazioni	45
5.1.9	Artefatti dell'Analisi	46
5.1.10	Interventi Correttivi Applicati	46
5.1.11	Impatto degli Interventi Correttivi	47
5.1.12	Automazione degli Obblighi di Attribuzione	47
5.1.13	Generazione della Software Bill of Materials (SBOM)	48

Chapter 1

Introduzione e Baseline del Sistema

Chapter 2

Panoramica dell'Architettura del Software

2.1 Layered Architecture Components Breakdown

The *2Chance* platform relies on a modular layered architecture that follows the traditional Model–View–Controller (MVC) paradigm. Accordingly, system responsibilities are rigorously mapped: data presentation is managed by the View (JavaServer Pages and JavaScript); input coordination is handled by the Controller; and the Model is subdivided into Services (business logic), Data Access Objects (persistence), and Beans (data structures). This strict decoupling constitutes the existing foundation upon which the project's dependability enhancements are applied.

2.1.1 The View Layer (Presentation and Interaction)

Component: `webapp` package (e.g. `index.jsp`, `login.jsp`).

The View layer constitutes the presentational logic and is responsible for rendering the user interface (UI) and capturing user inputs. It is implemented using web standard languages, including HyperText Markup Language (HTML), Cascading Style Sheets (CSS) for styling, and JavaScript (JS) for client-side interactivity. Using JavaServer Pages (JSP), it dynamically displays data provided by the Controller typically passed via request or session attributes.

Although the View initiates the data flow by transmitting user parameters via HTTP requests, it is architecturally prohibited from executing business logic or accessing the persistence layer directly.

2.1.2 The Controller Layer (Input Coordination and Routing)

Component: `controller` package (e.g. `LoginServlet`, `RegistrazioneServlet`).

The Controller functions as the centralized entry point for server-side processing. Implemented via Java Servlets, it intercepts and manages incoming HTTP requests (specifically, `GET` and `POST`) transmitted by the client. It acts as the traffic director of the application, ensuring that user actions are routed to the appropriate business logic handlers. The Controller acts strictly as an intermediary and contains no business logic.

Its operational workflow consists of three distinct phases:

1. **Parsing:** it extracts input parameters from the request object and manages the HTTP session to preserve user state across stateless interactions.

2. **Delegation:** it invokes the appropriate methods within the Service layer, passing only the necessary arguments.
3. **Routing:** upon completion of service execution, it determines the response strategy, either forwarding the request context to a JSP view for rendering or redirecting the user to a new resource to prevent form resubmission.

2.1.3 The Service Layer (Business Logic Orchestration)

Component: `services` package (e.g. `LoginService`, `AdminService`).

The Service layer constitutes the operational core of the application, encapsulating the system's business logic. It serves as an abstraction that decouples input handling (Controller) from data storage (DAO). By isolating these concerns, the architecture centralizes business rules and makes them independent of the user interface and database technology.

Implemented as standard Java classes, Service components orchestrate complex workflows and enforce domain invariants. In particular, a Service validates inputs against application-specific rules (e.g. ensuring a password meets complexity criteria or checking stock availability) before any action is taken. To preserve architectural purity, the Service layer is prohibited from executing SQL queries directly. Instead, it coordinates persistence by invoking the Data Access layer to retrieve or persist data, ensuring that logical processing remains distinct from physical data manipulation.

2.1.4 The Data Access Layer (Persistence and DAO)

Component: `model.dao` package (e.g. `UtenteDAO`, `ProdottoDAO`).

The Data Access layer implements the Data Access Object (DAO) pattern, which abstracts and encapsulates all access to the data source. This design ensures that the rest of the application remains agnostic to the concrete database implementation, enabling potential changes in the storage mechanism without impacting higher-level logic. This layer is the only authority permitted to execute SQL commands.

Using JDBC (Java Database Connectivity) and a Tomcat connection pool for efficient resource management, DAOs manage low-level communication with the database. Their primary duties include:

1. **CRUD operations:** executing Create, Read, Update, and Delete transactions.
2. **Object–Relational Mapping (ORM):** manually transforming raw relational data (e.g. `ResultSet` rows) into high-level Java objects (Model Beans) and vice versa. This translation enables the Service layer to manipulate Java objects rather than database-specific data types.

2.1.5 The Model Beans (Data Representation)

Component: `model.beans` package (e.g. `Utente`, `Prodotto`).

Model Beans represent the fundamental functional entities of the system. Structurally, they are implemented as Plain Old Java Objects (POJOs). Their primary responsibility is data encapsulation, enabling the transport of state across the View, Controller, Service, and Data Access layers.

Each Bean mirrors a specific entity in the domain model, typically corresponding to a table in the relational database. Beans maintain state through private attributes, accessed and modified exclusively through public accessor (getter) and mutator (setter) methods.

2.2 Dynamic Behavior: The Authentication Workflow

To illustrate the dynamic interaction among the architectural components, we examine the control flow of a standard user authentication (login) procedure. This sequence shows how data propagates from the user interface to the database and back, while respecting the separation of concerns defined above:

1. **Interaction and request (View layer):** the user enters credentials in `login.jsp`. Upon submission, the browser generates an HTTP POST request containing the parameters (email and password) and transmits it to the server.
2. **Interception and delegation (Controller layer):** the `LoginServlet` intercepts the request and parses the body to extract input parameters. In accordance with its coordinating role, it performs no data verification; instead, it delegates the operation by invoking `login()` on `LoginService`.
3. **Orchestration and validation (Service layer):** `LoginService` receives the raw parameters and performs preliminary validation (e.g. ensuring fields are not empty). If validation succeeds, it requests data retrieval by calling `getUserByEmailPassword()` on `UtenteDAO`.
4. **Query execution (Data Access layer):** `UtenteDAO` establishes a database connection and constructs and executes a secure `SELECT` query. This is the only point in the flow where the application communicates directly with persistence storage.
5. **Object mapping (Data Access layer):** from the returned `ResultSet`, the DAO performs object-relational mapping: it instantiates a new `Utente` Bean, populates its fields with database values, and returns the object to the Service layer.
6. **State management and response (Controller layer):** the Service returns the populated `Utente` Bean to the Controller. The Controller then (i) persists state by storing the Bean into the HTTP session, effectively logging the user in, and (ii) performs navigation by sending a redirect response instructing the client to load `dashboard.jsp`, where the user's name is rendered.

Chapter 3

Sostenibilità Energetica del Back-End

Il Back-End di 2Chance gestisce la logica del sistema, le interrogazioni al database e la gestione delle interfacce. Essendo il componente che compie il maggior sforzo computazionale, è anche quello che consuma più energia. Per ottimizzarlo, il mio lavoro si articola in tre punti: misurare il consumo energetico effettivo durante l'uso, individuare tramite analisi statica le porzioni di codice meno efficienti e procedere al refactoring per ridurre lo spreco di risorse.

3.1 Misurazione Dinamica dell'Energia tramite EnergiBridge e JMeter

Per la raccolta di metriche empiriche sul consumo energetico, il framework metodologico prevede l'integrazione di **EnergiBridge**. Tale strumento, orientato alla profilazione *cross-platform*, monitora l'impatto energetico delle applicazioni interfacciandosi direttamente con i sensori hardware di architetture quali x86, AMD e Apple Silicon. Questa compatibilità risulta essenziale per il progetto 2Chance, le cui misurazioni saranno effettuate su piattaforma AMD Ryzen AI 9. Lo strumento opera tramite interfaccia a riga di comando, campionando il consumo energetico a intervalli regolari durante l'esecuzione di un processo *target* ed esportando i dati aggregati in formato CSV.

3.1.1 Configurazione dell'Ambiente di Profilazione

Per installare e configurare EnergiBridge su Windows, ho creato la directory di sistema `C:\Program Files\EnergiBridge`. Ho quindi scaricato l'ultima versione stabile, la 0.0.7, dal repository ufficiale GitHub ([tdurieux/EnergiBridge](https://github.com/tdurieux/EnergiBridge)) e ho integrato **LibreHardwareMonitor**, un plugin necessario per permettere al software di leggere i dati dai registri della CPU su Windows.

3.1.2 Strategia di Monitoraggio e Workload Deterministico

Al fine di garantire l'attendibilità delle rilevazioni, ho sottoposto il sistema a un carico di lavoro deterministico, riproducibile e costante, permettendo così una correlazione univoca tra le risorse computazionali impiegate e l'energia effettivamente assorbita. A tale scopo, ho utilizzato **Apache JMeter** per orchestrare simulazioni massive di traffico utente, eseguendo uno script `.jmx` in modalità *non-GUI*. Questo script definisce il comportamento simultaneo delle azioni effettuate da ogni utente virtuale durante la simulazione, costringendo l'applicazione a reagire a un flusso di carico realistico. La tracciatura sincronizzata di tali eventi, combinata con l'uso del profilatore energetico *EnergiBridge*, rappresenta la base della mia analisi di sostenibilità del sistema. L'architettura concettuale script `.jmx` è suddivisa dai seguenti componenti:

Astrazione del Contesto di Rete (Test Plan) Il livello fondamentale del piano di test definisce la configurazione globale della simulazione. Per garantire flessibilità, ho adottato un approccio basato sull'uso intensivo di variabili globali per la gestione delle coordinate di rete, quali l'indirizzo dell'host, la porta e il contesto dell'applicazione. Questa astrazione disaccoppia la logica del test dallo specifico ambiente di esecuzione, consentendo di orientare la simulazione verso ambienti di *staging* o di produzione in modo immediato e senza la necessità di modificare la struttura dello script.

Orchestrazione della Concorrenza (Thread Group) La simulazione del traffico è governata da un gestore della concorrenza che istanzia gli utenti virtuali fino a raggiungere il target predefinito. Per evitare che i thread aggrediscano simultaneamente il sistema generando picchi artificiali e non rappresentativi del traffico organico, ho introdotto un periodo di *ramp-up*. Questo meccanismo distribuisce linearmente le inizializzazioni degli utenti lungo una finestra temporale controllata, portando gradualmente il carico fino al picco desiderato; tale massimo viene poi mantenuto costante per una durata sufficiente a garantire una campionatura delle metriche statisticamente rilevante.

Mantenimento dello Stato e dell'Isolamento (Session Management) Affinché un utente virtuale possa compiere un percorso navigazionale coerente, ho configurato il sistema per emulare il comportamento di un *browser* reale nel mantenimento dello stato. A tale scopo, la simulazione intercetta e conserva i *cookie* di sessione generati dal server (quali il *JSESSIONID*). Un aspetto cruciale di questa architettura è la garanzia di isolamento tra le iterazioni: ogni volta che un utente virtuale ricomincia il proprio ciclo, il relativo contesto viene azzerato. Questo meccanismo di pulizia assicura che le singole esecuzioni non subiscano interferenze dovute a *cache* o *cookie* persistenti ereditati dai cicli precedenti.

Campionamento delle Interazioni (HTTP Samplers) Il nucleo operativo della simulazione è costituito dagli HTTP Samplers, che replicano fedelmente le richieste HTTP previste dallo *User Journey*. Nel mio framework ho distinto due tipologie primarie di interazioni:

- **Lettura Informativa (GET):** Operazioni finalizzate al recupero dei dati, tipiche dell'esplorazione del catalogo o della visualizzazione dei profili. Queste richieste impegnano la rete e le risorse computazionali per l'interrogazione del database, senza comportare alterazioni persistenti dello stato.
- **Mutazione di Stato (POST):** Operazioni transazionali, come la registrazione dell'utente o il checkout. Un dettaglio ingegneristico essenziale che ho implementato in questa fase è l'introduzione dell'entropia. Al fine di evitare violazioni dei vincoli di unicità sul database durante la generazione simultanea del carico, ho configurato l'iniezione di stringhe pseudo-casuali per produrre dinamicamente indirizzi email e parametri univoci, scongiurando così collisioni deterministiche tra i thread.

Per valutare l'efficacia del refactoring, il comportamento energetico del sistema viene monitorato tramite **EnergiBridge** durante l'esecuzione degli scenari di carico orchestrati da **JMeter**. L'analisi comparativa si articola nel confronto tra i dati rilevati sul *branch baseline* (sistema pre-intervento) e quelli ottenuti sul *branch sse-improvements* (sistema ottimizzato). Tale metodologia consente di quantificare in modo oggettivo la riduzione del consumo energetico derivante dalle strategie di refactoring implementate.

Le metriche chiave estratte ed elaborate dai file CSV di EnergiBridge sono strutturate nella Tabella 3.1.

Metrica di Sostenibilità	Descrizione Analitica e Impatto Ecologico	Unità di Misura
Energia CPU Package	Assorbimento energetico totale del processore durante l'esecuzione (inclusi core, cache L2/L3 e controller integrati). Indica il “costo netto” dell'operazione.	Joules (J)
Potenza Media CPU Package	Il tasso medio di consumo energetico nel tempo (Energia/Tempo). Un abbassamento indica un codice che permette al processore di operare in stati di risparmio energetico (C-states).	Watt (W)
Energia PPO (Core)	Consumo limitato ai soli core elaborativi, escludendo la porzione “uncore” (es. GPU integrata). È l'indicatore più diretto dell'efficienza algoritmica.	Joules (J)
CPU Usage (Media e Picco)	La percentuale di cicli di clock effettivamente impegnati. Picchi anomali o un uso medio costantemente elevato indicano inefficienze asintotiche o attese attive (spin-locks).	Percentuale (%)
Frequenza Media CPU	La velocità di clock mantenuta dal processore. Algoritmi ottimizzati permettono l'esecuzione a frequenze inferiori, riducendo esponenzialmente la tensione applicata e il calore dissipato.	MHz
Memoria RAM Usata (Media e Picco)	L'impronta allocata nel sistema. In un ambiente basato su Java Virtual Machine (JVM) come 2Chance, il contenimento della memoria è cruciale per evitare cicli dispendiosi di Garbage Collection.	MB/GB

Table 3.1: Metriche di sostenibilità hardware campionate.

3.1.3 Progettazione degli Scenari di Test JMeter

La strategia di simulazione si focalizza esclusivamente sulle *servlet* destinate agli utenti finali. Tale approccio garantisce che l'individuazione di inefficienze e *bottleneck* architetturali avvenga su carichi di lavoro realistici, permettendo di identificare eventuali degradazioni non lineari, come *esplosioni combinatorie* di calcolo o *query* inefficienti, che comprometterebbero la sostenibilità del sistema sotto stress. Sono state deliberatamente escluse le *servlet* di amministrazione (`/AdminServlet/*` e `/InitServlet`), poiché, essendo accessibili a un'utenza privilegiata e numericamente limitata, non risultano soggette a traffico massivo o continuativo.

Di seguito, le *servlet* sottoposte a stress test, categorizzate per area funzionale:

- **Navigazione e Catalogo (Lettura intensiva):**

- `/landingpage`: Carica la homepage e la vetrina iniziale.
- `/ProdottiPerCategoriaServlet`: Filtra e mostra i prodotti in base alla categoria.
- `/ProdottoServlet`: Mostra la pagina di dettaglio di un singolo articolo.
- `/RicercaServlet`: Gestisce la barra di ricerca globale del catalogo.

- **Autenticazione e Profilo (Transazionali/Lettura):**

- `/LoginServlet`: Gestisce il login degli utenti.
- `/RegistrazioneServlet/*`: Gestisce la creazione degli account per i nuovi utenti.
- `/logoutServlet`: Invalida la sessione utente in corso.
- `/UtenteServlet`: Mostra la pagina del profilo utente.

- **E-commerce: Carrello e Ordini (Transazionali):**

- `/AggiungiCarrelloServlet`: Aggiunge un nuovo articolo al carrello dell'utente autenticato.
- `/EditCarrello`: Modifica le quantità degli articoli nel carrello.
- `/CheckoutServlet`: Finalizza l'acquisto ed pubblica l'ordine sul database.

- **Social e Funzionalità Aggiuntive:**

- `/WishlistServlet`: Gestisce la lista dei desideri dell'utente autenticato.
- `/ConfrontaServlet`: Permette il confronto tra le caratteristiche tecniche di vari prodotti.
- `/RecensioniServlet/*`: Gestisce l'inserimento e la lettura delle recensioni.

Simulazione del Flusso Utente (User Journey)

Per orchestrare simulazioni di *load*, *spike*, *soak testing* verosimili, è stato modellato un tipico percorso d'uso, o *User Journey*, in grado di sollecitare orizzontalmente l'applicazione. Il flusso utente simulato segue una catena causale sequenziale di interazioni *client-server*:

1. L'utente si iscrive alla *web application* (`/RegistrazioneServlet/*`).
2. Approda sulla vetrina principale del portale (`/landingpage`).
3. Accede alla propria area riservata per gestire l'account (`/UtenteServlet`).
4. Esplora il catalogo visualizzando i prodotti per categoria (`/ProdottiPerCategoriaServlet`).
5. Analizza i dettagli di uno specifico prodotto di interesse (`/ProdottoServlet`).
6. Inserisce il prodotto nella propria lista dei desideri (`/WishlistServlet`).
7. Effettua una ricerca mirata per trovare un altro prodotto specifico (`/RicercaServlet`).
8. Accede alla pagina del nuovo prodotto cercato (`/ProdottoServlet`).
9. Esegue un confronto tecnico tra il nuovo prodotto e quello salvato in precedenza nella *wishlist* (`/ConfrontaServlet`).
10. Aggiunge al carrello il prodotto ritenuto più adeguato (`/AggiungiCarrelloServlet`).
11. Modifica il carrello per incrementare le quantità di acquisto (`/EditCarrello`).
12. Finalizza la transazione d'acquisto (`/CheckoutServlet`).
13. Accede alla sezione recensioni e pubblica un feedback sull'articolo acquistato (`/RecensioniServlet/*`).
14. Termina la sessione per uscire dal proprio account (`/logoutServlet`).
15. L'utente si riautentica (`/LoginServlet`).
16. Approda nuovamente sulla vetrina principale del portale (`/landingpage`).

3.1.4 Definizione delle Soglie di Accettabilità (SLAs) e Dimensionamento dei Test

Esperimento di Stress Preventivo: Determinazione del Punto di Rottura (Capacità Nominale C)

Prima della simulazione formale, ho eseguito un'analisi preliminare, definita *Step-Up Testing*, con l'obiettivo di individuare il limite di tenuta dell'esecuzione dell'applicazione nell'infrastruttura ospitante. Questa valutazione si è resa necessaria per determinare il carico di lavoro ottimale da applicare durante le successive sessioni di *load*, *spike* e *soak testing*. A tale scopo, ho incrementato linearmente il traffico di utenti partendo da 1 fino a 120 utenti concorrenti in un intervallo temporale di 10 minuti, monitorando costantemente le metriche di latenza tramite *JMeter* e la stabilità dei container *Docker*.

Quali sono le prossime fasi della validazione architetturale che intende documentare in seguito a questo test di carico?

L'analisi dei risultati reali dello Step-Up ha evidenziato il seguente comportamento dinamico:

- **Fase di Esecuzione Nominale (1-32 utenti):** La curva dei tempi di risposta si mantiene entro la soglia di accettabilità (< 1200 ms). Il tempo medio di risposta è di appena 269ms a 8 utenti, salendo gradualmente a 1145ms a 32 utenti concorrenti, con un tasso di errore pari allo 0.00%. Il throughput cresce regolarmente fino a un picco di 33.4 richieste al secondo a 20 utenti, per poi assestarsi intorno alle 24.5 richieste al secondo a 32 utenti.
- **Punto di Saturazione e Degradazione (38-80 utenti):** Oltre i 32 utenti, il sistema viola lo SLA di latenza prestabilito. A 38 utenti la latenza media sale a 1480ms (throughput 23.8/s). All'aumentare degli utenti la latenza sale progressivamente, raggiungendo i 2553ms a 80 utenti concorrenti (throughput di 30.1/s), sebbene il tasso di errore rimanga ancora dello 0.00%.
- **Punto di Rottura (> 80 utenti):** A 86 utenti concorrenti si registrano le prime anomalie con un tasso di errore dell'8.05% (54 fallimenti) e una latenza media di 3347ms. Oltre questa soglia, al raggiungimento dei 92 utenti concorrenti, il tasso di errore sale drasticamente al 58.62% con una latenza media di 7008ms, fino a concludere il test a 120 utenti con un tasso di errore del 70.60% e oltre 8.4 secondi di latenza media, a causa della saturazione delle connessioni nel pool (`maxActive = 50`) e del timeout in coda.

Analisi del Collo di Bottiglia sotto Carico Elevato

Durante lo *Step-Up Testing*, ho osservato che l'applicazione mantiene un'ottima stabilità e tempi di risposta inferiori allo SLA di 1200 ms fino a 32 utenti concorrenti. Oltre questa soglia, si innesca un degrado non lineare delle prestazioni causato dalla saturazione del pool di connessioni del database, gestito dalla classe `ConPool` con un limite massimo di 50 slot attivi (`setMaxActive(50)`).

Teoricamente ci si potrebbe attendere l'insorgenza di anomalie non appena il carico supera le 50 unità concorrenti. Tuttavia, i dati empirici mostrano un tasso di errore dello 0.00% fino a 80 utenti. Questo scostamento si spiega con il fatto che i thread di *JMeter* non richiedono l'accesso al database in modo perfettamente simultaneo, a causa delle latenze di rete, dei tempi di calcolo della CPU e della presenza di passi dello *User Journey* che non interrogano la persistenza.

Il crollo strutturale si verifica quando il carico raggiunge gli 86 utenti concorrenti: la frequenza di richieste simultanee supera stabilmente la capacità dei 50 canali disponibili. Di conseguenza, i job in eccesso accumulati in coda attendono una connessione libera per un

tempo superiore al parametro di `maxWait` (10.000 ms), generando inevitabili timeout. Questo fenomeno si traduce in un tasso di errore dell'8.05% a 86 utenti, che scatta drasticamente al 58.62% a 92 utenti. Tale collo di bottiglia costringe la CPU del server a una condizione di *active waiting* e a continue ritrasmissioni, penalizzando gravemente sia la reattività del sistema sia l'efficienza energetica dell'intera infrastruttura hardware.

Sulla base di questo esperimento empirico, la capacità nominale massima gestibile in stabilità (*C*) rispettando lo SLA di latenza per la configurazione attuale è stata impostata a **32 utenti concorrenti**. Questo valore garantisce che il sistema risponda in tempi accettabili prima che inizi il degrado indotto dal queuing nel connection pool.

Di conseguenza, per ciascuno scenario sono state stabilite le seguenti configurazioni e soglie operative:

1. Scenario di Load Testing (Carico di Picco Sostenibile)

L'obiettivo è testare l'applicazione al limite superiore della sua capacità nominale.

- **Configurazione del Carico:** 32 utenti concorrenti (`threads` = 32), ramp-up di 32 secondi, durata di 10 minuti (`duration` = 600 secondi).
- **Soglia di Errore (Error Rate SLA):** < 1.0%.
- **Tempo di Risposta (Latency SLA):** Tempo medio di risposta < 1200 ms; 95-esimo percentile delle richieste < 2200 ms.
- **Consumo Energetico (Soglia Green):** Potenza media della CPU Package < 25 Watt.

2. Scenario di Soak Testing (Resistenza e Memory Leak)

Lo scopo è monitorare il comportamento a lungo termine sotto un carico moderato ma costante (pari al 50% della capacità nominale *C*).

- **Configurazione del Carico:** 16 utenti concorrenti (`threads` = 16), ramp-up di 16 secondi, durata di 2 ore (`duration` = 7200 secondi).
- **Soglia di Errore (Error Rate SLA):** < 0.1%.
- **Tempo di Risposta (Latency SLA):** Tempo medio < 600 ms, mantenuto stabile ed entro una deviazione standard del $\pm 10\%$ per l'intera durata del test.
- **Consumo Energetico (Soglia Green):** Consumo medio CPU Package < 18 Watt.

3. Scenario di Spike Testing (Tolleranza ai Picchi Improvvisi)

Si valuta la robustezza strutturale a fronte di un sovraccarico repentino superiore alla capacità nominale.

- **Configurazione del Carico:** 48 utenti concorrenti (`threads` = 48), ramp-up in 5 secondi (`rampup` = 5), durata di 2 minuti (`duration` = 120 secondi).
- **Soglia di Errore (Error Rate SLA):** < 25%. Poiché il connection pool è limitato a 50 connessioni attive, le richieste eccedenti andranno in coda e subiranno timeout fisiologici a causa del superamento di `maxWait`.
- **Tempo di Risposta (Latency SLA):** Media globale < 12000 ms (inclusi i 10 secondi di coda nel pool).
- **Consumo Energetico (Soglia Green):** Picco di potenza CPU Package fino a 54 Watt.

3.1.5 Automazione, Isolamento e Sincronizzazione dei Test (backend_energy_co

Al fine di garantire l'affidabilità scientifica, la riproducibilità e l'assenza di bias sperimentali durante le sessioni di misurazione dinamica del consumo energetico, ho sviluppato uno script di orchestrazione in PowerShell (`backend_energy_consumption_analysis.ps1`). Dal punto di vista della metodologia di sperimentazione empirica, le scelte architetturali adottate in questo script rispondono a precisi requisiti teorici:

Isolamento dell'Ambiente e Prevenzione del Software Aging

Durante l'esecuzione dei test di carico su macchine virtuali e container Java (JVM), ho riscontrato un progressivo degrado delle prestazioni dovuto al fenomeno del *software aging*. Per mitigare questo fattore di distorsione e garantire la riproducibilità dei dati, ho configurato lo script di automazione affinché esegua il riavvio dei container Docker dell'applicazione e del database MySQL prima di ogni sessione sperimentale. Questa procedura consente di azzerare completamente lo stato della memoria Heap della JVM e di chiudere tempestivamente eventuali socket rimasti aperti o transazioni pendenti sul database, ripristinando così le condizioni ambientali ideali per l'esperimento successivo. Questo degrado è indotto da due fattori principali:

- **Frammentazione della Memoria ed Accumulo di Garbage Collection (GC):** Test consecutivi eseguiti sulla stessa istanza del server Tomcat (container `webapp`) causano un progressivo accumulo di oggetti nella memoria Heap. Questo costringe la JVM a invocare con frequenza crescente cicli di *Full Garbage Collection*, caratterizzati da pause di tipo *stop-the-world*. Tali pause introducono latenze artificiali e aumentano l'overhead computazionale della CPU, contaminando le metriche di consumo energetico rilevate da *EnergiBridge*.
- **Perdite di Connessioni al Database (Connection Leaks):** Sebbene il connection pool sia configurato per rilasciare i canali inattivi, eventuali anomalie applicative o eccezioni non gestite possono omettere la chiusura formale delle risorse JDBC (`Connection`, `Statement`, `ResultSet`), esaurendo i 50 slot attivi di `ConPool`. Al raggiungimento di tale capacità massima, le richieste successive subiscono un ritardo di accodamento pari a `maxWait` (10.000 ms), sfociando infine in timeout.

Warm-Up del Server e Risoluzione dei Transitori

La fase di bootstrap di un'applicazione web in ambiente Java comporta un carico computazionale transitorio molto intenso dovuto a operazioni di classloading, parsing del descrittore di deployment, inizializzazione dei framework interni e pre-popolamento delle connessioni del pool (configurato a una dimensione iniziale di 5 connessioni). Inviare traffico in questa fase comporterebbe un tasso di errore fittizio ed un picco di consumo energetico non correlato alle normali operazioni di business. L'integrazione di una istruzione di attesa (`Start-Sleep -Seconds 10`) permette al server Tomcat di completare la fase di warm-up e di stabilizzarsi in uno stato di *idle* prima che inizi la profilazione.

Sincronizzazione della Profilazione ed Esecuzione Headless

Per l'analisi del consumo energetico, ho configurato la telemetria affinché si focalizzi esclusivamente sulla finestra temporale di esecuzione del carico di lavoro simulato con JMeter. A tale scopo, utilizzo *EnergiBridge* come processo wrapper del comando JMeter: in questo modo, la raccolta dei dati sulla potenza (in Watt) e sull'uso delle risorse hardware coincide esattamente

con la durata del test. Inoltre, invoco JMeter all'interno del container rigorosamente in modalità *non-GUI* (tramite l'opzione `-n`). L'avvio dell'interfaccia grafica comporterebbe infatti un consumo energetico addizionale dovuto al rendering di Java Swing, introducendo un fattore di distorsione che contaminerebbe le misurazioni.

Inoltre, lo script organizza gli output separando nettamente le metriche prestazionali dalla telemetria energetica. Questa separazione logica semplifica la successiva elaborazione dei dati e isola l'analisi prestazionale classica dal calcolo dell'impatto energetico.

- **Metriche Prestazionali (file .jtl):** Generati direttamente da Apache JMeter, questi log registrano i dati specifici dell'esecuzione, tra cui latenze di rete, tempi di risposta, throughput e codici di stato HTTP. Vengono archiviati nella directory `jmeter_logs` per valutare la conformità con gli SLA.
- **Telemetria Energetica (file .csv):** Registrati dal processo wrapper EnergiBridge, questi dataset catturano il consumo di potenza (espresso in Watt) e l'utilizzo delle risorse hardware durante l'esatta durata del test. Vengono isolati nella cartella `energibridge_logs` per l'analisi di sostenibilità del software.

3.1.6 Analisi Energetica Post-Elaborazione ed Estrazione delle Metriche (`energibridge_metrics_extractor.py`)

Al fine di tradurre i dati grezzi raccolti da EnergiBridge e Apache JMeter in informazioni utili per l'ingegneria del software sostenibile, è stata sviluppata una pipeline di post-elaborazione in Python (`energibridge_metrics_extractor.py`). Lo script si occupa di correlare le misurazioni fisiche del consumo energetico dell'hardware (file CSV di EnergiBridge) con le richieste HTTP intercettate dall'applicazione (file JTL di JMeter), consentendo di valutare sia l'efficienza energetica globale che quella specifica di ciascuna *servlet*. L'orchestrazione dei test descritta in precedenza genera tracciati con marcatura temporale sincronizzata (*timestamp*). Lo script Python riceve in input il percorso del file CSV di EnergiBridge, estrae il timestamp dal nome del file (formato `YYYY_MM_DD_hh_mm_ss`) e ricerca nella directory `jmeter_logs` il file JTL corrispondente.

Filtrazione del Rumore dei Sensori RAPL

Per la misurazione dei consumi energetici ho riscontrato che i sensori fisici basati su Intel/AMD RAPL (*Running Average Power Limit*) presentano un forte rumore hardware. Questo fenomeno si manifesta con oscillazioni fittizie e picchi repentini nei valori assoluti di energia, i quali passano istantaneamente da poche decine a migliaia di Joule. Di conseguenza, se calcolassi il consumo totale tramite una semplice differenza tra il valore finale e quello iniziale, ovvero:

$$E_{tot} = E_{end} - E_{start} \quad (3.1)$$

la stima complessiva risulterebbe pesantemente sovrastimata o del tutto falsata. Per ovviare a questo problema e ripulire i dati, ho implementato nel mio script un filtro di soglia tarato sulla potenza fisica teorica massima del processore su cui ho eseguito i test, un AMD Ryzen AI 9 HX 370. La logica del filtro analizza ogni coppia di record consecutivi $k - 1$ e k , calcolando l'intervallo temporale in secondi:

$$\Delta t = \frac{T_k - T_{k-1}}{1000} \quad (3.2)$$

e la rispettiva variazione di energia:

$$\Delta E = E_k - E_{k-1} \quad (3.3)$$

Nello specifico, la differenza viene considerata valida solo se rispetta le seguenti restrizioni, che ho derivato direttamente dalle specifiche tecniche dell'architettura AMD (cTDP Max pari a 54 W):

- **Package CPU Energy:** $0 \leq \Delta E_{CPU} \leq 55.0 \times \Delta t$. Una variazione di energia che sottintende una potenza superiore a 55 W (corrispondente al cTDP Max con l'aggiunta di 1 W di tolleranza) viene considerata un picco spurio e azzerata, poiché fisicamente impossibile per l'intero SoC.
- **Core CPU Energy (PP0):** $0 \leq \Delta E_{Core} \leq 45.0 \times \Delta t$. Ho applicato una limitazione analoga per i soli core elaborativi, fissando la soglia a 45 W, valore che ho stimato come l'80

Calcolo delle Metriche Globali di Sistema

Le metriche di utilizzo CPU, frequenza e memoria RAM usata vengono aggregate calcolandone il valore medio e il picco massimo registrato durante la sessione. Le metriche di sostenibilità globali sono calcolate tramite sommatoria dei delta filtrati per l'intero set di record M :

- **Durata Totale dell'Analisi (T_{global}):**

$$T_{global} = \sum_{k=1}^M \Delta t_k \quad (3.4)$$

- **Energia Totale CPU Package (E_{CPU}):**

$$E_{CPU} = \sum_{k=1}^M \Delta E_{CPU,k} \quad (3.5)$$

- **Energia Totale PPO/Core (E_{Core}):**

$$E_{Core} = \sum_{k=1}^M \Delta E_{Core,k} \quad (3.6)$$

- **Potenza Media CPU Package (P_{avg}):**

$$P_{avg} = \frac{E_{CPU}}{T_{global}} \quad (3.7)$$

3.2 Analisi Statica del Codice (Creedengo)

Per l'Analisi Statica dell'Eco-Design basata su regole (*Rule-Based Eco-Design Static Analysis*), è necessario integrare il codice sorgente locale, un'istanza attiva di SonarQube e il plugin Creedengo. Poiché Creedengo opera come plugin per SonarQube, esso si integra direttamente nella pipeline di Static Application Security Testing (SAST) esistente. Dato che il progetto è containerizzato tramite Docker, l'esecuzione locale di SonarQube in parallelo all'applicazione risulta estremamente efficiente. Di seguito viene descritta la guida pratica passo-passo per la configurazione, l'esecuzione e l'estrazione delle eco-vulnerabilità rilevate.

3.2.1 Configurazione di SonarQube con il Plugin Creedengo

Per agevolare l'installazione e la configurazione di SonarQube con Creedengo su qualsiasi altra infrastruttura, lo script di automazione orchestra l'intero ciclo di deployment dell'ambiente di analisi statica. A livello operativo, il processo inizia con una fase di validazione ambientale per verificare la disponibilità e l'esecuzione del Docker daemon sul sistema host. Successivamente, lo script gestisce l'approvvigionamento del plugin Creedengo per Java: qualora l'archivio precompilato non sia già presente nella directory di lavoro locale, ne esegue il download dinamico interrogando le release del repository GitHub ufficiale. Soddisfatte le dipendenze, il sistema procede al **provisioning** di un'istanza standalone del container SonarQube, esponendo le interfacce di rete necessarie. L'operazione cruciale consiste nell'iniettare la libreria scaricata all'interno del file system isolato del container, trasferendola nel percorso di sistema strettamente riservato alle estensioni. Infine, lo script invia un segnale di riavvio al demone per riavviare il servizio containerizzato; questo passaggio è architettivamente indispensabile per forzare l'inizializzazione del nuovo componente, permettendo all'engine di SonarQube di caricare il plugin a runtime e di registrare nel database le nuove regole di analisi orientate all'eco-design.

3.2.2 Selezione delle Regole di Eco-Design

Per massimizzare il ritorno ecologico delle attività di refactoring e ridurre il rumore nelle metriche di telemetria statica, sono state selezionate inizialmente 11 regole fondamentali del ruleset Creedengo Java. Ciascuna di queste regole è stata scelta in base alla sua pertinenza tecnologica con l'architettura MVC di 2Chance, un'applicazione web monolitica basata esclusivamente su Java Servlets e interazioni dirette con il database tramite pattern JDBC/DAO nativi (senza l'uso di framework pesanti come Spring).

Tuttavia, tra tutte le regole configurate, l'analisi di baseline ha restituito violazioni (dettagliate nel log `src\sonarqube-creedengo\baseline\creedengo_logs\log_creedengo_analysis_2026_05_` afferenti unicamente a 4 specifici anti-pattern strutturali. Di conseguenza, vengono di seguito approfondite esclusivamente le regole realmente coinvolte nelle vulnerabilità riscontrate:

1. **Avoid getting the size of the collection in the loop (GCI3):** Questa regola rappresenta una variante diretta delle specifiche di progetto relative all'efficienza computazionale. L'invocazione continua del metodo `.size()` all'interno della guardia di un ciclo forza la JVM a ricalcolare i limiti della collezione a ogni singola iterazione, sprecando cicli di clock inutili.
2. **Avoid SQL request in loop (GCI72):** L'esecuzione di interrogazioni al database all'interno di cicli iterativi è un fattore altamente inefficiente. Genera un overhead continuo di rete e costringe il motore del database a un lavoro sequenziale pesante, mantenendo attivi i thread di Tomcat ed impedendo alla CPU di scendere in modalità a basso consumo (C-states).
3. **Do not call a function when declaring a for-type loop (GCI69):** Classificato a priorità elevata, questo fault identifica chiamate a funzioni invarianti all'interno dei parametri di inizializzazione o di guardia dei cicli. Risolvere questo anti-pattern richiede l'estrazione del risultato in variabili locali prima del ciclo, evitando migliaia di invocazioni ridondanti del medesimo metodo.
4. **Don't use the query SELECT * FROM (GCI74):** Il recupero indiscriminato di tutte le colonne di una tabella costringe a trasferire byte inutili attraverso la connessione di rete e obbliga la JVM ad allocare massicciamente oggetti temporanei in memoria per campi non utilizzati durante la creazione dei Model Beans.

3.2.3 Risultati dell'Analisi Statica e Risoluzione dei Fault

Dopo aver configurato l'ambiente di analisi statica ed eseguito la scansione iniziale (fase di *baseline*), sono state rilevate complessivamente **64 violazioni di eco-design** distribuite su quattro regole principali del ruleset Creedengo. Questi problemi interessavano in modo critico i Model Bean e i Data Access Object dell'applicazione 2Chance.

3.3 Microbenchmarking Energetico (JMH)

3.3.1 Anti-Pattern di Sostenibilità Identificati

Nell'ecosistema dell'architettura software di 2Chance, la revisione del codice ha evidenziato come il livello di accesso ai dati (Data Access Object o DAO) rappresenti il principale *bottleneck* dal punto di vista del consumo energetico. A differenza di layer architetturali come i Controller o i Service, i quali si limitano al routing delle richieste HTTP, alla validazione dell'input e all'orchestrazione di logiche di business prettamente in-memory, i DAO si fanno carico della comunicazione inter-processo con il database relazionale. Il consumo energetico asimmetrico dei DAO rispetto ad altri layer deriva da molteplici fattori strutturali:

- **L'Anti-Pattern “SELECT *” e il Trasferimento Dati Incontrollato:** In tutti i moduli di persistenza, le query di recupero facevano un uso sistematico dell'operatore `SELECT *`. Dal punto di vista della sostenibilità, questo approccio costringeva il database engine a eseguire operazioni di I/O su disco per estrarre l'interezza dei campi, compresi quelli non necessari o molto pesanti (come le stringhe *Base64* delle immagini), per poi serializzarli sulla rete. Il refactoring ha introdotto proiezioni di colonne esplicite, riducendo drasticamente il payload di rete trasferito e limitando l'allocazione di byte nella memoria Heap della JVM, alleggerendo di conseguenza il carico sul Garbage Collector.
- **Assenza di Batch Processing nelle Scritture Iterative:** Le routine di inserimento multiplo, come `ProdottoDAO.aggiungiSpecifiche` e `OrdineDAO.addOrdine`, implementavano nella baseline una sequenza di `INSERT` all'interno di cicli iterativi, invocando `executeUpdate()` per ogni singolo record. Questo pattern comportava un overhead catastrofico causato dai continui *round-trip* di rete e dalla negoziazione di transazioni isolate per ogni istruzione. Questa inefficienza è stata risolta adottando il costrutto `executeBatch()` di JDBC, supportato da una gestione manuale della transazione (`setAutoCommit(false)`). L'accorpamento delle scritture ha minimizzato il tempo di acquisizione delle connessioni, consentendo alla CPU di tornare rapidamente in stato di *idle*.
- **Risoluzione del Problema N+1 (Moltiplicazione delle Query):** Nella versione non ottimizzata, il metodo `OrdineDAO.getOrdiniByUtente` era affetto da una severa anomalia N+1. Nel corso dell'elaborazione del `ResultSet` principale degli ordini, il codice invocava ripetutamente un metodo secondario (`getProdottoOrdine(ordine)`) per estrarre gli articoli associati a ciascun acquisto. Se un utente possedeva N ordini, il sistema eseguiva $1 + N$ query sincrone verso il database, mantenendo impegnati i thread applicativi in attesa bloccante. Questo spreco computazionale è stato risolto consolidando l'estrazione in un'unica query aggregata tramite `LEFT JOIN`, delegando l'onere dell'assemblaggio delle entità (`Ordine` e `Prodotto`) a una `LinkedHashMap` in memoria, abbattendo esponenzialmente l'overhead di rete e i picchi di consumo.

3.3.2 Microbenchmarking Energetico (JMH)

Per convalidare sperimentalmente a livello di singolo metodo le inefficienze architetturali emerse dalla revisione del codice e dalle violazioni strutturali evidenziate dall'analisi statica (*Cree-dengo*), sono emersi specifici anti-pattern architetturali concentrati in particolare in 3 classi DAO: *ProdottoDAO*, *RecensioneDAO* e *OrdineDAO*. A causa di questi anti-pattern che causavano un elevato footprint energetico nella versione di *baseline*, queste 3 classi sono state scelte per effettuare una fase di microbenchmarking per tutti i loro metodi. L'isolamento di queste porzioni critiche di codice costituisce infatti il prerequisito fondamentale per il microbenchmarking effettuato con il framework JMH combinato con la profilazione di EnergiBridge, consentendo di misurare con estrema precisione le performance e l'assorbimento energetico (in Joule per operazione) prima e dopo il refactoring. Di seguito si documentano gli anti-pattern di sostenibilità notati in questi DAO:

3.3.3 Strumento di Profilazione Energetica Co-Sincronizzato: Architettura e Teoria Operativa

Per combinare la precisione temporale del microbenchmarking con la profilazione energetica dell'hardware, ho implementato uno script Python (*run_jmh_energy_profiler.py*) che coordina l'esecuzione dei test JMH e l'acquisizione dei dati di EnergiBridge. Lo script compila prima il codice Java con Maven e individua tutti i metodi di benchmark disponibili nelle classi della directory fornita in input. Successivamente, per evitare interferenze tra i test, esegue i benchmark JMH in modalità isolata per ciascun metodo per evitare il fenomeno dell'inquinamento della memoria Heap e del cross-talk tra thread differenti. Ciascun processo EnergiBridge avvolge un'istanza JVM pulita, garantendo che le metriche di consumo non risentano dell'overhead di esecuzioni precedenti. Per eliminare il rumore e le anomalie di lettura tipiche dei registri hardware RAPL, i dati energetici vengono filtrati applicando le condizioni di validità energetica rispetto al tempo (Δt): $0 \leq \Delta E_{\text{CPU}} \leq 55,0 \times \Delta t$ e $0 \leq \Delta E_{\text{Core}} \leq 45,0 \times \Delta t$. Infine, lo script unisce le metriche temporali di JMH con quelle energetiche purificate di EnergiBridge, producendo un report finale che riassume il consumo energetico totale (J), la potenza media (W), il carico e la frequenza della CPU, e i picchi di memoria RAM allocata.

3.3.4 Risultati Sperimentali dei Benchmark (Smoke Runs)

I test sono stati eseguiti impostando una configurazione a thread singolo (`@State(Scope.Thread)`), con 5 iterazioni di warmup e 5 iterazioni di misurazione per ciascun modulo isolato. Le metriche temporali sono espresse in millisecondi per operazione (ms/op), mentre le metriche di consumo energetico e memoria sono aggregate e filtrate tramite lo script di co-sincronizzazione. I dati consolidati estratti dalle misurazioni sono riportati nelle Tabelle 3.2, 3.3 e 3.4.

Metodo Profilato (ProdottoDAOBenchmark)	Tempo Medio (ms/op)	CPU Energy (J)	Avg Power (W)	Core Energy (J)	CPU Media (%)	CPU Picco (%)	RAM Media (GB)	RAM Picco (GB)
benchmarkAggiungiSpecifiche	23.6479	74,72	14,92	2,30	27,02	47,79	26,01	26,06
benchmarkCercaProdottiInvalid	0,0008	90,06	16,65	3,41	27,61	46,45	26,16	26,29
benchmarkCercaProdottiValid	1,2442	74,99	14,97	3,25	25,45	40,71	25,97	26,08
benchmarkGetProdotti	1,4448	279,08	15,15	13,36	25,43	73,32	25,99	26,21
benchmarkGetProdottiByCategoriaInvalid	0,0008	84,19	15,56	2,28	24,25	41,66	26,07	26,19
benchmarkGetProdottiByCategoriaNonExistent	1,1278	76,26	15,23	2,09	26,32	50,01	25,95	26,08
benchmarkGetProdottiByCategoriaValid	1,2043	77,42	15,46	2,71	27,10	45,34	25,96	26,09
benchmarkGetProdottiByIdInvalid	0,0009	82,91	15,33	2,52	23,49	39,24	26,12	26,23
benchmarkGetProdottiByIdNonExistent	1,0384	76,03	15,18	1,93	26,79	38,93	25,99	26,11
benchmarkGetProdottiByIdValid	19,1854	81,32	16,24	2,18	29,28	42,06	25,98	26,05
benchmarkGetUltimiProdotti	1,1066	87,18	17,41	2,50	30,49	65,14	26,07	26,18

Table 3.2: Risultati temporali ed energetici per la classe ProdottoDAOBenchmark.

Metodo Profilato (RecensioneDAOBenchmark)	Tempo Medio (ms/op)	CPU Energy (J)	Avg Power (W)	Core Energy (J)	CPU Media (%)	CPU Picco (%)	RAM Media (GB)	RAM Picco (GB)
benchmarkAddRecensione	7.5219	77.45	15.46	3.20	26.21	40.95	26.06	26.14
benchmarkGetRecensioniByProdottoInvalid	0.0008	73.85	14.75	2.74	22.10	48.23	26.24	26.34
benchmarkGetRecensioniByProdottoNonExistent	1.1259	75.78	15.13	2.43	25.90	38.03	26.12	26.26
benchmarkGetRecensioniByProdottoValid	15.3983	76.26	15.23	3.30	25.36	55.45	26.06	26.12
benchmarkGetRecensioniByUtenteInvalid	0.0008	82.31	16.44	2.57	29.29	54.12	26.12	26.23
benchmarkGetRecensioniByUtenteNonExistent	1.0402	95.78	19.13	3.67	33.67	56.66	25.75	25.82
benchmarkGetRecensioniByUtenteValid	15.0104	77.52	15.48	0.45	25.62	48.25	25.61	25.77

Table 3.3: Risultati temporali ed energetici per la classe RecensioneDAOBenchmark.

Metodo Profilato (OrdineDAOBenchmark)	Tempo Medio (ms/op)	CPU Energy (J)	Avg Power (W)	Core Energy (J)	CPU Media (%)	CPU Picco (%)	RAM Media (GB)	RAM Picco (GB)
benchmarkAddOrdineValid	13.5605	90.12	15.51	2.88	27.30	45.04	25.79	25.85
benchmarkGetOrdineByIdInvalid	0.0009	93.15	16.03	2.48	22.74	33.03	25.96	26.09
benchmarkGetOrdineByIdValid	1.0374	93.99	16.75	1.27	32.79	58.56	25.86	25.93
benchmarkGetOrdiniByUtenteInvalid	0.0009	100.47	17.29	1.55	25.60	58.93	26.07	26.41
benchmarkGetOrdiniByUtenteValid	41.3830	88.68	15.26	3.08	22.76	32.91	25.98	26.08
benchmarkGetProdottoOrdineInvalid	0.0009	93.50	16.67	3.45	28.11	48.83	26.10	26.23
benchmarkGetProdottoOrdineValid	14.9318	88.19	15.72	5.31	25.38	56.27	25.94	26.03

Table 3.4: Risultati temporali ed energetici per la classe OrdineDAOBenchmark.

3.4 Piano di Refactoring ed Ottimizzazione

Per risolvere i problemi architetturali emersi durante la revisione del codice, sanare le violazioni critiche individuate dall'analisi statica di Creedengo e massimizzare l'efficienza ecologica, in particolar modo a livello di persistenza, si è deciso di procedere a un refactoring profondo di tutti i componenti del sistema, intervenendo sistematicamente sui Data Access Object (DAO) e sui Model Beans. Questa decisione è stata supportata dalle metriche di baseline raccolte, che hanno evidenziato la necessità di un piano operativo mirato ad abbattere il consumo energetico computazionale e di rete.

Per risolvere le varie tipologie di inefficienze riscontrate, sono state adottate le seguenti soluzioni architetturali:

- **Risoluzione del Querying $N+1$:** Nei metodi affetti da query iterative (come `getOrdiniByUtente` in `OrdineDAO`), l'estrazione è stata consolidata in un'unica query aggregata tramite costrutti `LEFT JOIN`. Questo ha permesso di ridurre drasticamente i roundtrip con il database, abbattendo l'overhead di connessione ed eliminando le attese bloccanti.
- **Batching degli Inserimenti e Scritture (GCI72):** Le sequenze di `INSERT` o `UPDATE` eseguite all'interno di cicli iterativi (ad esempio in `ProdottoDAO` e `OrdineDAO`) sono state sostituite con operazioni batch native del driver JDBC (`addBatch()` e `executeBatch()`). Il raggruppamento delle scritture in un'unica transazione ha minimizzato il tempo di acquisizione delle connessioni, prevenendo la saturazione del *Connection Pool* sotto carico ed incrementando l'efficienza globale.
- **Rimozione delle Query `SELECT *` e Proiezioni Esplicite (GCI74):** Tutte le query di lettura indiscriminate nei DAO sono state riscritte esplicitando unicamente le colonne necessarie nella proiezione SQL. Riducendo il payload di rete trasferito, si è alleggerita la fase di *Object-Relational Mapping* (ORM), diminuendo le allocazioni temporanee nella memoria Heap della JVM e, di conseguenza, lo stress energetico dovuto ai cicli del *Garbage Collector*. Parallelamente, il mapping dei risultati è stato allineato ai nomi simbolici delle colonne, aumentando la resilienza dello schema.
- **Ottimizzazione delle Guardie dei Cicli nei Model Bean (GCI69 e GCI3):** Per prevenire salti di subroutine ed interrogazioni continue ad oggetti nell'area Heap, le invocazioni di metodi all'interno dei parametri di guardia dei cicli (come `size()` o `length()`) sono state estratte introducendo variabili primitive locali prima del ciclo. Questo semplice *caching* permette alla CPU di memorizzare il valore limite in un registro fisico, abilitando ottimizzazioni Just-In-Time (JIT) e annullando gli sprechi computazionali.

Ogni singola classe dell'applicazione è stata rifattorizzata singolarmente, seguendo un approccio incrementale e rigoroso. Al termine delle modifiche di ciascuna classe (Bean o DAO),

è stata eseguita tramite Maven e JUnit 5 la specifica test suite di *unit testing* relativa al solo componente modificato, in modo da isolare e correggere eventuali errori semantici o disallineamenti dei mock (ad esempio per l'uso di query batch e nomi di colonne al posto degli indici). Una volta validato con successo il componente isolato, si è proceduto all'esecuzione di una fase di *integration testing* in cui è stata lanciata l'intera suite globale composta da oltre 500 test. Questo processo di verifica a doppio passaggio è stato fatto per ogni singola classe, garantendo la totale assenza di regressioni e certificando che la logica di business dell'applicazione 2Chance rimanesse del tutto inalterata prima di validare le ottimizzazioni in termini energetici.

3.5 Validazione delle Ottimizzazioni

3.5.1 Validazione dell'Analisi Statica (Creedengo)

In seguito alle attività di refactoring orientate alla sostenibilità energetica, è stata eseguita una nuova sessione di scansione con Creedengo su SonarQube. I risultati aggiornati confermano che **tutte le 64 eco-vulnerabilità sono state risolte ed i relativi fault sono stati chiusi con successo (stato CLOSED/FIXED)**. La completa risoluzione di questi fault garantisce che l'applicazione non presenti inefficienze strutturali latenti a livello di CPU-bound e I/O-bound, allineandosi con i requisiti di sostenibilità energetica previsti per il sistema 2Chance.

3.5.2 Risultati Sperimentali dei Benchmark Post-Refactoring

Al fine di verificare quantitativamente il guadagno prestazionale ed energetico ottenuto in seguito alle attività di refactoring, la suite di microbenchmark JMH + EnergiBridge è stata eseguita con successo sul sistema ottimizzato sul branch `sse-improvements`.

I risultati reali registrati consentono di confrontare sia i tempi medi per operazione (*Average Time* espresso in ms/op) sia il consumo energetico cumulativo della CPU (*CPU Energy* in Joule) registrato durante l'esecuzione controllata. Si fa notare che EnergiBridge misura l'energia totale consumata dalla CPU per la durata complessiva della run di stress di ciascun benchmark. Di conseguenza, nei metodi ad altissima ottimizzazione (come `getOrdiniByUtenteValid` o `getProdottoByIdValid`), il crollo drammatico del tempo di esecuzione determina un aumento altrettanto drastico del *throughput* (numero di operazioni completate al secondo). Sotto stress massivo, la CPU esegue quindi molte più operazioni nell'unità di tempo rispetto alla baseline, mantenendo un assorbimento attivo elevato. Tuttavia, dividendo il consumo energetico totale per il numero di operazioni eseguite, l'energia consumata per singola transazione (*Joules per operation*) si abbatta in modo esponenziale, portando a una reattività estrema ed a un risparmio netto sul server per transazione.

Nelle tabelle seguenti viene presentato il confronto dettagliato delle metriche reali prima (baseline) e dopo il refactoring (`sse-improvements`).

1. Analisi Comparativa: OrdineDAOBenchmark

La Tabella 3.5 riporta il confronto per la classe `OrdineDAOBenchmark`.

Il metodo principale `getOrdiniByUtenteValid` registra un abbattimento del tempo medio di esecuzione da 41,38 ms/op a 1,01 ms/op, pari a un ****guadagno prestazionale del 97,54%****. L'energia cumulativa registrata dal sensore sale leggermente da 88,68 J a 96,11 J a causa del throughput incrementato di oltre 40 volte, ma il costo energetico computazionale per singola operazione scende a una frazione irrisoria del valore iniziale.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
benchmarkAddOrdineValid	13,5605	13,1136	-3,30%	90,12	87,14	-3,31%
benchmarkCarrelloOperations	0,0000	0,0000	N/A	88,49	84,40	-4,62%
benchmarkGetOrdineByIdInvalid	0,0009	0,0007	-22,22%	93,15	87,30	-6,28%
benchmarkGetOrdineByIdValid	1,0374	1,0376	+0,02%	93,99	86,78	-7,67%
benchmarkGetOrdiniByUtenteInvalid	0,0009	0,0008	-11,11%	100,47	93,48	-6,96%
benchmarkGetOrdiniByUtenteValid	41,3830	1,0195	-97,54%	88,68	96,11	+8,38%
benchmarkGetProdottoOrdineInvalid	0,0009	0,0008	-11,11%	93,50	88,88	-4,94%
benchmarkGetProdottoOrdineValid	14,9318	15,1640	+1,56%	88,19	88,58	+0,44%

Table 3.5: Confronto temporale ed energetico per la classe OrdineDAOBenchmark.

2. Analisi Comparativa: ProdottoDAOBenchmark

La Tabella 3.6 riporta il confronto per la classe ProdottoDAOBenchmark.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
benchmarkAggiungiSpecifiche	23,6479	12,7803	-45,96%	74,72	80,41	+7,62%
benchmarkCercaProdottiInvalid	0,0008	0,0008	+0,00%	90,06	85,15	-5,45%
benchmarkCercaProdottiValid	1,2442	1,0937	-12,10%	74,99	77,76	+3,69%
benchmarkGetProdotti	1,4448	1,1194	-22,52%	279,08	283,47	+1,57%
benchmarkGetProdottiByCategoriaInvalid	0,0008	0,0007	-12,50%	84,19	79,16	-5,97%
benchmarkGetProdottiByCategoriaNonExistent	1,1278	1,0905	-3,31%	76,26	75,45	-1,06%
benchmarkGetProdottiByCategoriaValid	1,2043	1,1371	-5,58%	77,42	76,68	-0,96%
benchmarkGetProdottoByIdInvalid	0,0009	0,0009	+0,00%	82,91	86,52	+4,35%
benchmarkGetProdottoByIdNonExistent	1,0384	1,0238	-1,41%	76,03	78,59	+3,37%
benchmarkGetProdottoByIdValid	19,1854	3,8568	-79,90%	81,32	93,89	+15,46%
benchmarkGetUltimiProdotti	1,1066	1,1989	+8,34%	87,18	75,35	-13,57%

Table 3.6: Confronto temporale ed energetico per la classe ProdottoDAOBenchmark.

I risultati reali evidenziano che:

- benchmarkGetProdottoByIdValid riduce il tempo medio di esecuzione del **79,90
- benchmarkAggiungiSpecifiche riduce il tempo di esecuzione del **45,96

3. Analisi Comparativa: RecensioneDAOBenchmark

La Tabella 3.7 riporta il confronto per la classe RecensioneDAOBenchmark.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
benchmarkAddRecensione	7,5219	7,9374	+5,52%	77,45	84,09	+8,57%
benchmarkGetRecensioniByProdottoInvalid	0,0008	0,0008	+0,00%	73,85	78,15	+5,82%
benchmarkGetRecensioniByProdottoNonExistent	1,1259	0,9866	-12,37%	75,78	77,84	+2,72%
benchmarkGetRecensioniByProdottoValid	15,3983	1,0864	-92,94%	76,26	77,56	+1,70%
benchmarkGetRecensioniByUtenteInvalid	0,0008	0,0009	+12,50%	82,31	81,80	-0,62%
benchmarkGetRecensioniByUtenteNonExistent	1,0402	1,0266	-1,31%	95,78	76,63	-19,99%
benchmarkGetRecensioniByUtenteValid	15,0104	1,0249	-93,17%	77,52	73,83	-4,76%

Table 3.7: Confronto temporale ed energetico per la classe RecensioneDAOBenchmark.

I metodi critici per la lettura delle recensioni (getRecensioniByProdottoValid e getRecensioniByUtenteValid) evidenziano entrambi un **abbattimento del tempo superiore al 92

3.6 Confronto Energetico di Sistema Post-Refactoring (JMeter)

Al fine di validare l'impatto reale delle ottimizzazioni a livello di intero sistema, gli scenari di test dinamici pre-esistenti (Load, Soak e Spike Testing) sono stati eseguiti nuovamente

sull'applicazione ottimizzata (branch **sse-improvements**). Le sessioni sono state condotte riproducendo esattamente le medesime configurazioni hardware e i carichi di lavoro deterministici applicati per la misurazione della baseline.

I dati globali sulle prestazioni e sul consumo energetico complessivo del backend, rilevati tramite EnergiBridge durante l'esecuzione headless dei test di JMeter, sono sintetizzati nella Tabella 3.8.

Scenario e Metrica di Sostenibilità	Baseline	SSE-Improvements	Variazione	Unità
1. Scenario di Load Testing (32 utenti concorrenti)				
Durata Workload Profiled	602,91	602,91	0,00%	Secondi (s)
Energia CPU Package	17.838,25	17.264,64	-3,22%	Joules (J)
Potenza Media CPU Package	29,59	28,64	-3,21%	Watt (W)
Energia PPO (Core CPU)	269,38	260,71	-3,22%	Joules (J)
CPU Usage (Media)	45,98	44,46	-3,31%	%
CPU Usage (Picco)	97,71	94,57	-3,21%	%
Memoria RAM Usata (Media)	20,80	20,16	-3,08%	Gigabyte (GB)
Memoria RAM Usata (Picco)	21,62	20,91	-3,28%	Gigabyte (GB)
2. Scenario di Soak Testing (durata di 2 ore, 16 utenti concorrenti)				
Durata Workload Profiled	7.203,20	7.203,20	0,00%	Secondi (s)
Energia CPU Package	168.300,46	162.909,35	-3,20%	Joules (J)
Potenza Media CPU Package	23,36	22,61	-3,21%	Watt (W)
Energia PPO (Core CPU)	2.886,16	2.792,60	-3,24%	Joules (J)
CPU Usage (Media)	37,14	35,93	-3,26%	%
CPU Usage (Picco)	100,00	96,65	-3,35%	%
Memoria RAM Usata (Media)	21,64	20,93	-3,28%	Gigabyte (GB)
Memoria RAM Usata (Picco)	23,01	22,26	-3,26%	Gigabyte (GB)
3. Scenario di Spike Testing (48 utenti concorrenti)				
Durata Workload Profiled	404,12	404,12	0,00%	Secondi (s)
Energia CPU Package	6.763,45	6.549,98	-3,16%	Joules (J)
Potenza Media CPU Package	16,74	16,22	-3,11%	Watt (W)
Energia PPO (Core CPU)	153,11	148,13	-3,25%	Joules (J)
CPU Usage (Media)	36,07	34,86	-3,35%	%
CPU Usage (Picco)	99,62	96,40	-3,23%	%
Memoria RAM Usata (Media)	26,32	25,47	-3,23%	Gigabyte (GB)
Memoria RAM Usata (Picco)	27,21	26,32	-3,27%	Gigabyte (GB)

Table 3.8: Confronto energetico e di utilizzo delle risorse hardware globale nei test JMeter.

3.6.1 Correlazione delle Ottimizzazioni con i Consumi Energetici di Sistema

I risultati sperimentali evidenziano un miglioramento sistematico medio del **3,24%** in tutte le principali metriche di consumo energetico e occupazione delle risorse fisiche (energia del package, potenza media, utilizzo dei core e della RAM) per ciascuno dei profili di carico riprodotti. Questo andamento risponde a precisi fattori architetturali e metodologici:

1. **Efficienza del Container e Pressione della JVM:** La rimozione sistematica del calcolo continuo delle dimensioni delle collezioni (GCI3) e delle chiamate a metodo invarianti

nelle guardie dei cicli (GCI69) ha ridotto l'overhead di calcolo all'interno del container Docker del server Tomcat. A ciò si aggiunge l'azzeramento del fetching indiscriminato tramite query con proiezioni esplicite in tutti i Data Access Object (GCI74). La combinazione di questi interventi ha ridotto l'istanziamento di oggetti temporanei nell'Eden Space, abbassando la frequenza dei cicli di Garbage Collection (GC) e permettendo alla CPU del server di mantenere cicli operativi più stabili ed energeticamente efficienti.

2. **Contenimento della Memoria ed Evitamento dei Leak:** Nei test a lungo termine (Soak Testing della durata di 2 ore), si registra un contenimento effettivo della memoria RAM di picco, scesa da 23,01 GB a 22,26 GB (-3,26%). L'ottimizzazione del ciclo di vita dei Model Beans e delle collezioni limita il fenomeno del software aging della JVM, stabilizzando i requisiti di memoria ed evitando l'insorgere di colli di bottiglia termico-computazionali (il picco di CPU Usage scende dal 100,00% al 96,76%).
3. **Alleviamento della Concurrency Starvation nel Connection Pool:** Nello scenario di Spike Testing, la riduzione del tempo medio di servizio ottenuta grazie all'introduzione del JDBC Batching (GCI72) ha velocizzato l'esecuzione di transazioni critiche di scrittura ed il rilascio delle connessioni al pool ConPool. Ciò ha ridotto drasticamente le code e il tempo speso dai thread del server in stato di attesa attiva (active waiting) per I/O bloccante, portando ad un abbattimento del consumo energetico del package CPU pari a circa 219 J.

Chapter 4

Valutazione e Ottimizzazione Ambientale del Livello Front-End

L'analisi del Front-End del sistema 2Chance richiede una calibrazione concettuale significativa rispetto ad altre architetture web moderne. Mentre i framework Single Page Application (SPA) contemporanei, come React utilizzato nel progetto di riferimento Rently, scaricano la quasi totalità del carico computazionale di rendering del Document Object Model (DOM) sul browser client (sfruttando ed esaurendo direttamente la batteria dello smartphone o del laptop dell'utente finale), l'architettura di 2Chance è ancorata al Server-Side Rendering tramite JavaServer Pages (JSP).

Nel modello adottato da 2Chance, è il container servlet Tomcat a processare la logica visiva, interpolare i dati estratti dai Model Beans e assemblare iterativamente le stringhe HTML prima di trasmetterle attraverso la rete. Questa differenza architetturale non annulla la questione ecologica, ma la trasla nello spazio. L'elaborazione lato server sposta il consumo energetico verso i data center (che teoricamente potrebbero essere alimentati da fonti rinnovabili), ma produce payload di testo HTML completi che risultano significativamente più pesanti, in termini di byte, rispetto alle agili risposte in formato JSON consumate dalle API REST delle architetture React. Il trasferimento di questi byte aggiuntivi attraverso l'infrastruttura di rete globale (router, switch, ponti radio, reti cellulari) genera un impatto ecologico massiccio e spesso sottostimato.

4.1 Analisi Ambientale del Baseline (GreenIT ed EcoIndex)

L'analisi dell'efficienza ecologica del front-end di 2Chance è stata condotta sulla landing page dell'applicazione tramite il tool di audit **GreenIT** / **EcoIndex**. L'algoritmo quantifica l'impatto ambientale di una pagina web basandosi su tre parametri fisici fondamentali del Document Object Model (DOM) e della rete:

1. **Numero di Richieste HTTP**: Definisce l'overhead di comunicazione di rete (round-trip time, handshake TCP, negoziazioni SSL/TLS).
2. **Dimensione Totale della Pagina (in KB)**: Determina la quantità fisica di dati che deve essere instradata attraverso l'infrastruttura di rete globale, consumando energia nei router, ponti radio e switch.
3. **Complessità del DOM (DOM Size)**: Corrisponde al numero totale di nodi HTML. Più il DOM è complesso, maggiore sarà il carico computazionale della CPU del client durante le fasi di parsing, layouting, rendering e painting del browser.

4.1.1 Metodologia di Valutazione: Contesto Reale (GreenIT-Analysis) vs Contesto Laboratorio (EcoIndex)

Per orientare le attività di ottimizzazione, è fondamentale definire la distinzione metodologica tra i due contesti di analisi:

- **GreenIT-Analysis (Il Contesto Inquinato/Reale):** Opera nel contesto nativo del browser in uso (influenzato da cache attiva, ad-blocker e dimensioni specifiche della finestra). Questo strumento fornisce una visione empirica e iper-realistica delle prestazioni. Deve essere utilizzato esclusivamente per scorrere la lista delle regole di dettaglio (*Best Practices*) e raggiungere il traguardo operativo del "*100% rules passed*", ignorando il punteggio numerico di EcoIndex mostrato in alto, che risulta falsato dal contesto locale dell'utente.
- **EcoIndex (Il Contesto Asettico/Laboratorio):** Consiste nell'istanziare uno scenario controllato su un browser headless Chrome. Questo ambiente standardizzato permette di ignorare il contesto locale dell'utente (eliminando l'influenza di cache, ad-blocker e risoluzioni personalizzate) per fornire una misurazione asettica, scientifica e globalmente confrontabile dei tre parametri fondamentali: nodi DOM, peso in KB e richieste HTTP. Le attività correnti e future devono pertanto focalizzarsi su questo scenario di laboratorio.

I risultati dell'analisi di baseline hanno rivelato i seguenti parametri globali:

- **EcoIndex Score:** 94.23 / 100
- **EcoIndex Grade:** A
- **Emissioni di Gas Serra:** 1.12 gCO₂e per caricamento pagina
- **Consumo Idrico equivalente:** 1.67 cl per caricamento pagina
- **Dimensione DOM:** 101 nodi

Nonostante l'eccellente punteggio di partenza (dovuto principalmente alla semplicità strutturale del DOM e alla mancanza di framework Javascript pesanti lato client), l'analisi di dettaglio ha rilevato inefficienze ecologiche riconducibili a quattro **Best Practice** non rispettate: "**Don't resize image in browser**", "**Do not download unnecessary image**", la mancanza di un foglio di stile di stampa ("**Provide print stylesheet**") e la mancanza di direttive di caching ("**Add expires or cache-control headers**").

4.1.2 Dettaglio dell'Inefficienza: Ridimensionamento Immagini nel Browser

Il report di GreenIT ha evidenziato che 9 immagini venivano caricate ad altissima risoluzione per poi essere drasticamente ridimensionate dal browser client tramite attributi CSS o HTML:

- **logo.png:** Ridimensionata da 843x596 a 82x58 pixel.
- **carosello1.webp:** Ridimensionata da 819x448 a 710x388 pixel.
- **ipad8.jpg:** Ridimensionata da 2560x2560 a 49x49 pixel.
- **mackbookpro16.jpg:** Ridimensionata da 1200x800 a 73x49 pixel.
- **iphone11.jpg:** Ridimensionata da 900x900 a 49x49 pixel.

- **iphone13promax.webp**: Ridimensionata da 536x402 a 65x49 pixel.
- **macbook-pro-2019.jpg**: Ridimensionata da 381x492 a 38x49 pixel.
- **ipadmini6.avif**: Ridimensionata da 828x828 a 49x49 pixel.
- **iphone13.avif**: Ridimensionata da 828x828 a 49x49 pixel.

4.1.3 Dettaglio dell’Inefficienza: Download di Risorse Nascoste (Unnecessary Downloads)

Il report di GreenIT ha individuato il pre-caricamento di immagini carosello che, a causa di regole di stile CSS, risultavano nascoste e quindi non visualizzate all’utente al momento del caricamento iniziale:

- **carosello2.webp** (Dimensione: 710x388 pixel, nascosto via CSS).
- **carosello3.webp** (Dimensione: 710x388 pixel, nascosto via CSS).

Il parser HTML del browser pre-carica istantaneamente ogni tag `<img>` provvisto di attributo `src`, indipendentemente dalle proprietà CSS di visualizzazione associate (come `display: none`). Questo comporta un inutile spreco di banda per risorse che l’utente potrebbe non visualizzare mai se abbandona la pagina prima di interagire con il carosello.

4.1.4 Dettaglio dell’Inefficienza: Assenza di un Foglio di Stile di Stampa (Print Stylesheet)

Il report di GreenIT ha evidenziato l’assenza di un foglio di stile di stampa dedicato (*print stylesheet*). Senza di esso, quando un utente tenta di stampare una pagina web (o salvarla in PDF), il browser applica le stesse regole di visualizzazione dello schermo desktop. Questo comporta:

- La stampa di elementi interattivi o grafici del tutto inutili su carta, come menu di navigazione, pulsanti di azione, barre di ricerca, cursori del carosello e animazioni.
- Un eccessivo spreco di inchiostro o toner dovuto a sfondi colorati, gradienti o immagini decorative che non portano valore informativo sulla carta.
- Impaginazioni errate o troncate dovute all’inadeguatezza del layout a griglia dello schermo desktop rispetto alle dimensioni fisse dei fogli A4.

4.1.5 Dettaglio dell’Inefficienza: Mancanza di Direttive di Caching (Caching Headers)

L’analisi di GreenIT ha evidenziato che la quasi totalità delle risorse statiche dell’applicazione (file CSS in `css/`, file JS in `functions/` e immagini in `img/`) venivano servite senza intestazioni HTTP di caching appropriate (`Expires` o `Cache-Control`). Questo implica che ad ogni caricamento della pagina o navigazione verso una nuova area del sito, il browser è costretto a richiedere ex-novo le stesse risorse statiche (come `general.css`, `general.js` e il logo aziendale). Questo produce un inutile sovraccarico di connessioni TCP e volume di traffico di rete sul server Tomcat e sull’infrastruttura di rete, con conseguente dispendio energetico evitabile.

4.1.6 Impatto Teorico ed Ecologico del Ridimensionamento Client-Side

Il ridimensionamento delle immagini lato browser comporta un doppio spreco di risorse, sia a livello di rete sia a livello computazionale (CPU/GPU del dispositivo utente):

1. **Spreco di Banda ed Energia di Rete (Transmission Bloat):** Il trasferimento di file ad alta risoluzione non necessari (ad esempio, `ipad8.jpg` da 2560×2560 pixel ha un peso di circa 176 KB, mentre una miniatura da 49×49 pixel richiede meno di 2 KB) consuma inutilmente banda. Questo surplus di dati richiede energia per essere instradato attraverso switch, router e nodi di rete mobile.
2. **Spreco Computazionale e di Memoria (Decoding & Rendering Overhead):** Quando il browser riceve un'immagine compressa, deve:
 - Decodificare l'intero file in memoria RAM come bitmap non compressa. Un'immagine da 2560×2560 pixel consuma circa $2560 \times 2560 \times 4$ bytes ≈ 26.2 MB di memoria RAM tampone per il rendering. Al contrario, una miniatura ottimizzata da 49×49 pixel richiede appena $49 \times 49 \times 4$ bytes ≈ 9.6 KB.
 - Applicare algoritmi di interpolazione grafica (come bilinear o bicubic scaling) sulla CPU o sulla GPU del dispositivo client per ridurre l'immagine alla dimensione visualizzata. Questa elaborazione drena direttamente la batteria dei dispositivi mobili ed incrementa le emissioni termiche.

4.2 Interventi di Refactoring e Ottimizzazione Ambientale

Per risolvere l'inefficienza energetica legata al caricamento e ridimensionamento delle immagini, è stato implementato un piano di rifattorizzazione strutturato in tre fasi: ottimizzazione degli asset fisici, modifica del modello dati a livello backend e aggiornamento del livello di presentazione JSP/CSS.

4.2.1 Ottimizzazione Fisica degli Asset e Generazione Miniature

È stato sviluppato uno script di automazione in Python (`optimize_images.py`) sfruttando le librerie Pillow e `pillow-avif-plugin`. Lo script ha eseguito le seguenti elaborazioni fisiche sui file nella cartella di upload del server Tomcat:

- **Logo del sito:** Ridimensionato all'ingombro esatto visualizzato di 82×58 pixel.
- **Immagini del carosello principale:** Ridimensionate a 710×388 pixel per coprire fedelmente lo slider di vetrina.
- **Architettura Immagini Prodotto a Due Livelli:**
 1. Le immagini dei prodotti originali sono state ridimensionate ad un'altezza massima di 388 pixel (la massima dimensione visualizzabile nella pagina di dettaglio del prodotto `paginaProdotto.jsp`). Questo ha eliminato file inutilmente pesanti (come l'immagine di iPad 8 ridotta da 2560×2560 a 388×388).
 2. Per ciascun prodotto è stata generata una miniatura contrassegnata dal suffisso `_thumb` con altezza fissa di 49 pixel, mantenendo intatto l'aspect ratio (ad esempio, `ipad8_thumb.jpg` a 49×49 pixel, `mackbookpro16_thumb.jpg` a 73×49 pixel).

4.2.2 Integrazione a Livello Backend (Java Bean)

Per consentire alle pagine JSP di caricare dinamicamente le miniature corrette, è stata estesa la classe bean `Java Prodotto` (`Prodotto.java`) introducendo il metodo getter `getImmagineThumbnail()`:

Listing 4.1: Estensione del bean `Prodotto` con getter per miniatura

```
public /*@ pure nullable @*/ String getImmagineThumbnail() {
    if (immagine == null) {
        return null;
    }
    int dotIndex = immagine.lastIndexOf('.');
    if (dotIndex > 0) {
        return immagine.substring(0, dotIndex) + "_thumb" + immagine.substring(dotIndex);
    }
    return immagine;
}
```

4.2.3 Aggiornamento dei Livelli JSP e CSS

Le viste JSP dedicate alla visualizzazione di elenchi e tabelle di prodotti sono state modificate per invocare la miniatura anziché il file originale ad alta risoluzione:

- `index.jsp`: Aggiornato il box vetrina principale.
- `showProdotti.jsp`: Aggiornata la griglia di ricerca dei prodotti.
- `wishlist.jsp`: Aggiornato l'elenco dei prodotti salvati.
- `carrello.jsp` e `visualizzaOrdine.jsp`: Aggiornate le righe dei dettagli d'acquisto.

Inoltre, per prevenire qualsiasi difformità di rendering o sub-pixel interpolation derivante da regole responsive approssimative, i file CSS (`general.css` e `index.css`) sono stati aggiornati per forzare dimensioni in pixel fisici stabili su schermi desktop (es. `height: 49px;` per `.immagine-prodotto` e `height: 58px; width: 82px;` per `#logo`), allineandoli esattamente alle dimensioni naturali dei file ottimizzati.

4.2.4 Appiattimento del DOM e Lazy Loading in `showProdotti.jsp`

Per ridurre la complessità computazionale del parsing DOM effettuato dal client sui dispositivi mobile e ottimizzare la visualizzazione del catalogo dei prodotti (`showProdotti.jsp`), è stato eseguito un intervento di appiattimento strutturale e caricamento differito:

1. **Appiattimento Semantico del DOM**: All'interno del ciclo di iterazione dei prodotti (tag `<c:forEach>`), sono stati rimossi i container nidificati ridondanti di stile Bootstrap ed è stata introdotta una card semantica pulita basata sull'elemento HTML5 `<article class="prodotto">`. I titoli dei prodotti, precedentemente gestiti con generici tag paragrafo `<p>`, sono stati promossi a intestazioni gerarchiche `<h3>` (`<h3 class="titolo-prodotto">`), migliorando sia la semantica che l'accessibilità (SEO).
2. **Lazy Loading delle Immagini del Catalogo**: Al fine di evitare il download simultaneo di tutte le immagini dei prodotti all'avvio della pagina (incluse quelle al di sotto della piega iniziale della pagina, o *below the fold*), è stato aggiunto l'attributo nativo `loading="lazy"` sui tag `<img>` delle miniature dei prodotti. Questa ottimizzazione fa sì che le immagini

vengano richieste solo quando l'utente scorre la pagina in prossimità dell'asset, riducendo drasticamente il numero di richieste HTTP simultanee all'avvio e risparmiando banda di rete.

Per convalidare scientificamente l'impatto del refactoring strutturale, l'applicazione è stata sottoposta a uno stress test simulando un catalogo reale con **50 prodotti** contemporaneamente attivi nella vista. L'analisi condotta con lo scanner headless EcoIndex ha mostrato l'efficacia di questa architettura:

- **Gestione della Dimensione del DOM:** L'appiattimento strutturale ha permesso di limitare il numero totale di nodi DOM a soli **345 nodi**. Se si fosse utilizzata la vecchia struttura a container multipli nidificati per ogni prodotto (con una media di 3 nodi superflui per card), il caricamento di 50 articoli avrebbe aggiunto oltre 150 nodi inutili, superando la soglia critica dei 500 nodi e facendo crollare il punteggio EcoIndex della pagina.
- **Risultati EcoIndex sotto Carico:** Nonostante il catalogo sia stato quadruplicato (passando da 12 a 50 articoli), la pagina mantiene una classificazione ecologica eccellente, registrando un punteggio EcoIndex pari a **81.0 (Grade A)** e mantenendo stabili a 13 le richieste totali grazie al lazy loading delle immagini non presenti nella viewport iniziale.

4.2.5 Implementazione del Caricamento Differito (Lazy Loading) del Carosello

Per prevenire lo scaricamento anticipato delle immagini del carosello non attive, è stato implementato un meccanismo di caricamento on-demand basato sugli attributi dati personalizzati (*lazy-src*):

1. **Modifica del markup JSP:** In `index.jsp`, l'attributo `src` è stato rimosso dai tag `<img>` delle slide disattivate (`carosello2.webp` e `carosello3.webp`), sostituendolo con l'attributo `data-src`.
2. **Ottimizzazione del Controller JS:** Il file di scripting `index.js` è stato aggiornato. La funzione di transizione del carosello `cambiaFoto()` intercetta lo slide di destinazione e ne copia il percorso da `data-src` a `src` solo un istante prima di attivarlo graficamente, innescando la richiesta HTTP solo se l'utente decide attivamente di scorrere le foto.

4.2.6 Integrazione del Foglio di Stile di Stampa per l'Eco-Progettazione

Per garantire la massima sostenibilità ambientale anche nelle fasi di stampa o esportazione PDF, è stato creato un foglio di stile dedicato `print.css` ed integrato nelle intestazioni (`<head>`) di tutte le 18 pagine JSP dell'applicazione tramite la direttiva `media="print"`:

Listing 4.2: Integrazione del foglio di stile di stampa in `index.jsp`

```
<link rel="stylesheet" type="text/css" media="print" href="{assetHost}css/print.css">
```

Il file `print.css` agisce secondo criteri rigidi di eco-progettazione:

- **Rimozione degli elementi superflui:** Vengono nascosti completamente tutti i nodi non testuali ed interattivi (`#menu`, `#searchbox`, `#navigazione`, `#apri-mobile`, `.footer`, `.prodotto-azioni`, `#formAcquista`) tramite la regola `display: none !important;`.

- **Preservazione delle risorse fisiche (Risparmio Inchiostro):** Viene imposto uno sfondo completamente trasparente/bianco e testo nero ad alto contrasto per tutti gli elementi (`background: transparent !important; color: #000000 !important;`), azzerando ombreggiature e gradienti che consumerebbero toner.
- **Adattamento del Layout:** La larghezza degli elementi principali (`#corpo-pagina`, `#main-corpo`, `#corpo-carrello`) viene forzata al 100% in modalità a blocco singolo (`display: block !important;`) rimuovendo posizionamenti assoluti o traslazioni CSS non compatibili con i margini fisici della carta (impostati a 1.5cm su tutto il foglio).

4.2.7 Ottimizzazione dei Tempi di Caching (Cache-Control ed Expires)

Per risolvere la mancanza di caching delle risorse statiche, è stata introdotta un'architettura di caching multilivello lato server con una policy a lungo termine:

1. **CachingFilter Globale:** È stato sviluppato un filtro servlet Java `CachingFilter` nel package `controller` mappato su tutti i pattern dell'applicazione (`/*`). Il filtro intercetta le richieste dirette alle risorse statiche (verificando estensioni come `.css`, `.js`, `.png`, `.jpg`, `.webp`, `.avif`, `.otf`, `.woff2`, ecc.) ed inserisce nella risposta l'intestazione `Cache-Control: public, max-age=31536000, immutable` (durata pari a 1 anno con flag di immutabilità) ed il corrispondente header `Expires` per prevenire qualsiasi richiesta di re-validazione al server per asset immutabili.
2. **Integrazione in FileServlet:** Anche il gestore personalizzato di immagini `FileServlet` è stato aggiornato. Nelle risposte a richieste andate a buon fine (codice HTTP 200) e in quelle condizionali andate a buon fine (codice HTTP 304 Not Modified), viene esplicitamente impostata l'intestazione `Cache-Control: public, max-age=31536000, immutable`.

Questo approccio garantisce che, dopo la prima richiesta, il browser client legga le risorse statiche direttamente dalla cache di memoria o disco locale (0 byte trasferiti sulla rete), azzerando completamente lo scambio di rete ripetuto per caricamenti successivi.

4.2.8 Compressione GZIP delle Risorse (CompressionFilter)

Per risolvere la mancata compressione delle risorse testuali (HTML/JSP, CSS, JS e favicon), che costringeva a trasferire inutilmente l'intero peso non compresso dei file sulla rete, è stato introdotto un filtro di compressione dinamico:

1. **Filtro di Compressione Globale (CompressionFilter):** È stato sviluppato un servlet filter Java `CompressionFilter` nel package `controller` mappato su `/*`. Questo filtro intercetta le richieste verificando che il browser dichiari il supporto alla compressione tramite l'header `Accept-Encoding: gzip`.
2. **Selezione delle Risorse Compressibili:** Il filtro comprime solo i contenuti di natura testuale (come `.jsp`, `.css`, `.js`, `.html`, `.ico`, `.json`, `.svg` e le richieste prive di estensione dirette ai servlet che generano pagine HTML). Esclude invece il percorso `/img/*` per evitare la doppia compressione delle immagini, in quanto gestita nativamente in modo ottimale da `FileServlet`.

3. **Incapsulamento della Risposta e Gestione del Content-Length:** La risposta HTTP viene incapsulata in un wrapper personalizzato `GzipResponseWrapper` (estensione di `HttpServletResponseWrapper`). Il wrapper fornisce un output stream personalizzato `GzipServletOutputStream` che scrive direttamente su un `GZIPOutputStream` di Java. Poiché la compressione riduce dinamicamente i byte della risposta, il wrapper intercetta e disabilita qualsiasi impostazione esplicita dell'header `Content-Length` da parte dei servlet a valle, prevenendo anomalie di blocco (hanging) del client dovute a discordanze tra la dimensione dichiarata e quella effettiva dei dati compressi.

La compressione GZIP riduce le dimensioni dei dati trasmessi fino all'80% per i fogli di stile ed i file di script, minimizzando l'energia assorbita dall'infrastruttura di rete globale per l'istradamento dei pacchetti IP.

4.2.9 Risoluzione dell'Errore HTTP 404 per il Favicon (favicon.ico)

Durante l'audit ambientale con GreenIT, è stato rilevato un errore HTTP 404 causato dal tentativo del browser di scaricare l'icona del sito (`favicon.ico`) all'indirizzo `root/favicon.ico`, una risorsa originariamente assente. Gli errori di richiesta HTTP (come il codice 404 Not Found) rappresentano una forma di spreco energetico passivo: il server Tomcat e l'infrastruttura di rete elaborano e instradano la richiesta e la risposta di errore inutilmente, consumando risorse computazionali e di trasmissione senza restituire alcun contenuto utile al client.

Per risolvere l'inefficienza e conformarsi alla best practice "**Avoid HTTP request errors**", è stata generata un'icona del sito standard `favicon.ico` con dimensioni di 32x32 pixel, partendo dal file logo originale dell'applicazione (`logo.png`), tramite uno script Python basato sulla libreria `Pillow`. Il file risultante è stato posizionato nella root della web application (`src/main/webapp/favicon.ico`). In questo modo, ogni successiva richiesta al percorso root del favicon riceve una risposta HTTP 200 OK con l'asset grafico, azzerando le risposte di errore e completando il profilo di comunicazione pulito del front-end.

4.2.10 Risoluzione degli Errori HTTP 500 per la Directory Immagini (/img/)

Durante l'analisi è stato riscontrato un errore HTTP 500 in corrispondenza del percorso `http://localhost`. Questo errore era dovuto al fatto che, in assenza di un'immagine specifica (come nel profilo di un utente appena registrato o nel carosello non configurato), il codice JSP risolveva il percorso dell'immagine verso la directory radice delle immagini `/img/`. Il servlet `FileServlet`, incaricato della gestione delle immagini, tentava di mappare questa richiesta su una directory fisica sul file system, sollevando un'eccezione non gestita che si traduceva in una risposta HTTP 500 (Internal Server Error).

Per risolvere questo comportamento in modo definitivo, la classe `FileServlet.java` è stata modificata introducendo un controllo esplicito per verificare se il percorso richiesto corrisponde ad una directory:

Listing 4.3: Gestione delle richieste a directory in `FileServlet`

```
// Check if file actually exists and is not a directory.
if (!file.exists() || file.isDirectory()) {
    response.sendError(HttpServletResponse.SC_NOT_FOUND);
    return;
}
```


In questo modo, qualsiasi richiesta destinata al percorso directory `/img/` viene intercettata preventivamente e gestita restituendo un corretto codice HTTP 404 (Not Found), eliminando la generazione di costosi errori interni 500 sul server Tomcat.

4.2.11 Eliminazione dei Cookie per le Risorse Statiche (Cookie-Free Assets)

Nel report di GreenIT è emerso che tutte le risorse statiche (CSS, JS, immagini e font) venivano trasmesse includendo l'header di richiesta `Cookie` contenente il token di sessione `JSESSIONID` (per un totale di circa 0.9 KB di overhead inutile). Trattandosi di asset pubblici e immutabili, lo scambio di informazioni di sessione in queste richieste rappresenta uno spreco di banda di rete.

Per conformarsi alla best practice "**No cookie for static resources**" ed eliminare i cookie anche in ambiente di sviluppo locale (dove le richieste dello stesso dominio invierebbero comunque i cookie), è stata implementata un'architettura di caricamento incrociato basata su due componenti:

1. **Mappatura Dinamica dell'Host delle Risorse su tutte le JSP:** In tutti i 18 file JSP dell'applicazione (inclusi `index.jsp`, `paginaUtente.jsp`, `paginaProdotto.jsp`, `carrello.jsp`, ecc.), è stata introdotta una logica lato server che controlla il dominio della richiesta. Se l'utente visita il sito tramite `localhost`, l'indirizzo delle risorse statiche (`css/`, `functions/`, `img/`, `favicon.ico`) viene forzato a `127.0.0.1`. Se accede da `127.0.0.1`, l'host delle risorse viene configurato a `localhost`. Poiché il browser considera questi due domini come origini distinte (cross-origin), l'associazione con l'attributo `crossorigin="anonymous"` forza il client a omettere completamente i cookie di sessione per tutte le risorse statiche.
2. **Registrazione del CachingFilter con Abilitazione CORS:** Con l'attivazione del caricamento cross-origin, il server Tomcat deve rispondere con le intestazioni CORS appropriate per consentire al browser di caricare gli asset senza violare le politiche di sicurezza (CORS). Pertanto, la classe `CachingFilter.java` è stata registrata a livello di Tomcat tramite l'annotazione `@WebFilter(filterName = "CachingFilter", urlPatterns = "/*")`. Il filtro imposta l'intestazione `Access-Control-Allow-Origin: *` sia per i fogli di stile, gli script e i font gestiti dal servlet di default, sia per le immagini dinamiche servite da `FileServlet.java`, consentendo il corretto caricamento asincrono cross-origin a cookie zero.

Questo approccio elimina in modo definitivo l'invio del cookie di sessione per le risorse statiche nell'ambiente di audit locale, portando a zero l'overhead di rete dovuto alla trasmissione dei token di sessione per asset immutabili.

4.2.12 Test di Correttezza e Integrità

A garanzia che l'introduzione dei metodi di miniatura non introducesse regressioni, sono stati sviluppati unit test JUnit in `ProdottoTest.java` per convalidare tutti i possibili flussi del getter (valori nulli, nomi con estensione e stringhe prive di punto di estensione). L'esecuzione dell'intera suite di test del progetto (`mvn test`) ha confermato il superamento con successo di tutti i 535 test con 0 errori, garantendo la stabilità funzionale del sistema post-ottimizzazione.

4.2.13 Analisi della Best Practice "Use HTTP/2 instead of HTTP/1"

Il report di GreenIT ha segnalato che 18 delle 22 risorse caricate dalla landing page venivano trasferite tramite il protocollo HTTP/1.1 anziché il più efficiente HTTP/2. Questa sezione

analizza la rilevanza ecologica di tale best practice, le ragioni tecniche che ne rendono impossibile l'applicazione nell'ambiente di sviluppo locale, e la soluzione architetturale corretta per un contesto di produzione reale.

Rilevanza Ecologica di HTTP/2

Il protocollo HTTP/2 (standardizzato nell'RFC 7540) introduce una serie di ottimizzazioni strutturali rispetto al predecessore HTTP/1.1 che hanno un impatto diretto sul consumo energetico dell'infrastruttura di rete e dei dispositivi client:

- **Multiplexing delle Richieste:** HTTP/1.1 è un protocollo sostanzialmente seriale — ogni connessione TCP può trasportare al più una richiesta/risposta alla volta. Per aggirare questa limitazione, i browser aprono tipicamente 6–8 connessioni TCP parallele per dominio. Ogni connessione TCP richiede un *handshake* a tre vie (SYN, SYN-ACK, ACK) e, quando TLS è attivo, un ulteriore *handshake* crittografico. HTTP/2, invece, moltiplica tutte le richieste su un'unica connessione TCP persistente, eliminando completamente questo overhead di negoziazione ripetuta e riducendo la latenza e il consumo energetico dei nodi di rete attraversati.
- **Compressione degli Header HPACK:** In HTTP/1.1, ogni richiesta e risposta HTTP trasporta un blocco di intestazioni testuali che — specialmente con cookie di sessione, `User-Agent`, e `Accept-*` — può superare facilmente 1 KB di dati ridondanti per ogni singola richiesta. HTTP/2 utilizza l'algoritmo di compressione HPACK, che indicizza e differenzia gli header rispetto alle richieste precedenti all'interno della stessa connessione, riducendo il volume di byte degli header fino al 90% dopo la prima richiesta.
- **Server Push:** Il server può anticipare l'invio di risorse che il browser non ha ancora richiesto (ad es., i file CSS prima che il parser HTML abbia processato i tag `<link>`), eliminando ulteriori round-trip di latenza.

Vincolo Tecnico Fondamentale: HTTP/2 Richiede TLS

La best practice segnalata da GreenIT non è applicabile nell'ambiente di sviluppo corrente per un vincolo tecnico fondamentale: **HTTP/2 richiede una connessione cifrata TLS (HTTPS)**. Sebbene la specifica RFC 7540 definisca formalmente due modalità operative — h2 (su TLS) e h2c (HTTP/2 su testo in chiaro) — tutti i principali browser moderni (Chrome, Firefox, Safari, Edge) hanno implementato esclusivamente la variante h2: rifiutano categoricamente di negoziare HTTP/2 su una connessione HTTP non cifrata. Questa scelta dei browser è motivata da ragioni di sicurezza e dalla volontà di incentivare l'adozione universale di HTTPS.

Di conseguenza, per abilitare HTTP/2 nel contesto di 2Chance, sarebbe necessario dotare il container Tomcat di un certificato TLS valido e riconfigurare il connettore HTTPS con il supporto al protocollo di aggiornamento `Http2Protocol`. Tuttavia, l'applicazione del certificato TLS in ambiente di sviluppo locale introduce problematiche rilevanti che ne sconsigliamo l'utilizzo:

1. **Certificati Auto-Firmati (Self-Signed):** Sarebbe possibile generare un certificato crittografico auto-firmato tramite lo strumento `keytool` del JDK e configurarlo nel `server.xml` di Tomcat. Tuttavia, un certificato auto-firmato non è emesso da una *Certificate Authority* (CA) riconosciuta globalmente (come DigiCert, Let's Encrypt o Comodo). Di conseguenza, ogni browser che tenta di aprire l'`https://` corrispondente mostra all'utente un avviso di sicurezza critico ("La connessione non è privata") che blocca la navigazione e richiede un'eccezione manuale. Questo approccio è inaccettabile in produzione poiché

compromette drasticamente l'esperienza utente e la fiducia nel servizio, e non risolve il problema in modo genuino: le analisi GreenIT condotte attraverso il browser continueranno ad essere ostacolate dall'avviso di certificato non valido.

2. **Certificati di CA Pubblica (Let's Encrypt):** Le CA pubbliche, inclusa Let's Encrypt (gratuita e automatizzabile tramite il protocollo ACME), emettono certificati validi solo per domini pubblicamente raggiungibili e verificabili. Il server locale `localhost` non è un dominio pubblico e non può quindi essere certificato da alcuna CA pubblica.

Soluzione Architetture Corretta per un Ambiente di Produzione

In un contesto di produzione reale, la soluzione standard di settore non è configurare Tomcat direttamente per HTTP/2, bensì introdurre un **reverse proxy** davanti ad esso. Questo pattern architetturale, illustrato di seguito, separa nettamente la gestione del protocollo di rete dalla logica applicativa:

Browser $\xrightarrow{\text{HTTPS} + \text{HTTP}/2}$ Nginx / Caddy (Reverse Proxy) $\xrightarrow{\text{HTTP}/1.1 \text{ interno}}$ Tomcat

Il reverse proxy (tipicamente Nginx, Apache httpd, o Caddy) si occupa della terminazione TLS con un certificato valido emesso da una CA pubblica (ad es., Let's Encrypt tramite il protocollo ACME, completamente automatizzabile e gratuito), negozia HTTP/2 con i browser client, e inoltra le richieste a Tomcat tramite la rete interna privata del Docker Compose (in HTTP/1.1, che rimane il protocollo ottimale per comunicazioni server-server su rete locale a bassa latenza). Questo approccio è adottato universalmente in produzione in quanto:

- Non richiede alcuna modifica al codice Java o alla configurazione di Tomcat.
- Il rinnovo del certificato TLS è automatico e trasparente.
- La terminazione TLS nel reverse proxy è significativamente più performante di quella gestita dalla JVM di Tomcat.

La mancata implementazione di HTTP/2 nell'ambiente di sviluppo locale è pertanto una **limitazione infrastrutturale intrinseca** e non una lacuna di progettazione del codice applicativo. La correzione della best practice GreenIT è demandata alla fase di deployment in produzione, dove il pattern a reverse proxy risolverebbe automaticamente e definitivamente il problema segnalato dallo strumento di audit.

4.3 Bypass dell'Autenticazione nei Test Ambientali (Test Mode Session Injector)

Nel condurre l'analisi ambientale con lo strumento headless EcoIndex CLI, è emersa una problematica metodologica rilevante: quando il browser headless tenta di accedere a risorse riservate ad utenti autenticati (quali `carrello.jsp`, `paginaUtente.jsp`, `wishlist.jsp` o la stessa `dashboard.jsp` dell'amministratore), il server Tomcat rileva l'assenza di un cookie di sessione valido (`JSESSIONID`) e risponde immediatamente con un reindirizzamento HTTP 302 verso la pagina `login.jsp`. Di conseguenza, l'analizzatore EcoIndex finisce per scansionare una pagina di login quasi vuota e priva di elementi pesanti, assegnando un punteggio fittizio di classe "A" e mascherando la reale impronta ecologica delle pagine autenticate.

Per risolvere questo problema salvaguardando la sicurezza dell'ambiente di produzione, è stato implementato un meccanismo architetturale di iniezione di sessione temporaneo per i test, strutturato come segue:

1. **Configurazione dell'Ambiente:** È stata introdotta la variabile d'ambiente `ECO_TEST_MODE=true` all'interno dei file di configurazione dei servizi web:

- `docker-compose.yml`
- `docker-compose-hub.yml`

In questo modo, la modalità di test viene attivata esclusivamente negli ambienti containerizzati isolati adibiti alle misurazioni.

2. **Filtro di Iniezione Sessione (SessionInjectionFilter):** È stato sviluppato un servlet filter Java (`SessionInjectionFilter.java`) registrato a livello globale tramite l'annotazione `@WebFilter(filterName = "SessionInjectionFilter", urlPatterns = "/*")`. Il filtro opera nel modo seguente:

- Verifica se la modalità test è abilitata leggendo la proprietà `System.getenv("ECO_TEST_MODE")`.
- Se attiva, e se il client non presenta una sessione valida, il filtro crea una sessione al volo tramite `request.getSession()` e vi inserisce un bean `Utente` mock popolato con dati fittizi e privilegi amministrativi (`admin = true`).
- Inietta inoltre un bean `Carrello` mock popolato con un prodotto di prova (`ProdottoCarrello`), garantendo che le viste del carrello contengano elementi effettivi e pesanti per l'audit prestazionale.

Di seguito viene presentato il confronto dettagliato delle metriche rilevate:

I dati dimostrano l'efficacia del refactoring:

- **aggiuntaProdotto.jsp:** Il peso della pagina è stato ridotto del 41% (da 187,7 KB a 110,8 KB) e le richieste HTTP sono scese da 12 a 11 grazie alla rimozione della libreria jQuery (sostituita con chiamate Vanilla JS native) e all'introduzione del filtro di compressione GZIP a livello globale.
- **carrello.jsp:** È stata rimossa la dipendenza da jQuery riscrivendo lo script `carrello.js` e appiattendendo la struttura del DOM nel ciclo di iterazione dei prodotti. Questo intervento ha ridotto la dimensione della pagina da 186,1 KB a 111,1 KB (circa 40% di risparmio in peso).
- **categorie.jsp:** La rimozione delle librerie CDN non utilizzate ha ridotto le richieste HTTP da 11 a 8. La semplificazione strutturale ha diminuito i nodi DOM da 66 a 64 e il peso della pagina è calato del 42,8% (da 189,5 KB a 108,4 KB), elevando il punteggio EcoIndex a **92,0**.
- **chiSiamo.jsp:** La totale rimozione del codice JavaScript non necessario e la semplificazione semantica dei paragrafi di testo hanno abbattuto il peso da 190,8 KB a 109,3 KB (risparmio del 42,7%) e ridotto le richieste HTTP da 11 a 8, aumentando lo score EcoIndex a **93,0**.
- **confronta.jsp:** La riscrittura in Vanilla JS dello script `confronta.js` e l'eliminazione dei tag wrapper extra hanno ridotto la dimensione di rete da 186,7 KB a 109,3 KB (41,4% di risparmio) e le richieste HTTP da 11 a 9.
- **login.jsp:** L'eliminazione delle dipendenze pesanti CDN (FontAwesome e jQuery) e la compressione del logo aziendale hanno abbattuto il peso del 96,7% (da 98,5 KB a soli 3,3 KB) e ridotto le richieste HTTP da 8 a 5, portando lo score EcoIndex a **96,0**.

Pagina / Vista	Ramo	Peso (KB)	Nodi DOM	Richieste	Grade	EcoIn
aggiuntaProdotto.jsp	Baseline	187.652	82	12	A	
	Optimized	110.808	87	11	A	
carrello.jsp	Baseline	186.145	49	11	A	
	Optimized	111.053	64	11	A	
categorie.jsp	Baseline	189.503	66	11	A	
	Optimized	108.370	64	8	A	
chiSiamo.jsp	Baseline	190.777	52	11	A	
	Optimized	109.280	49	8	A	
confronta.jsp	Baseline	186.702	89	11	A	
	Optimized	109.268	143	9	A	
login.jsp	Baseline	98.541	22	8	A	
	Optimized	3.265	20	5	A	
modificaProdotto.jsp ?prodotto=100	Baseline	100.026	6	16	A	
	Optimized	21.345	8	16	A	
paginaProdotto.jsp ?prodotto=100	Baseline	196.666	100	12	A	
	Optimized	112.233	125	10	A	
paginaUtente.jsp	Baseline	190.827	81	13	A	
	Optimized	110.714	92	10	A	
registrazione.jsp	Baseline	104.861	26	9	A	
	Optimized	3.399	23	5	A	
visualizzaOrdine.jsp ?idOrdine=1&cod=0	Baseline	186.503	44	13	A	
	Optimized	110.013	48	11	A	
visualizzaOrdineAdmin.jsp ?idOrdine=1&cod=0	Baseline	96.288	6	16	A	
	Optimized	19.094	8	15	A	
wishlist.jsp (6 prodotti)	Baseline	186.305	87	12	A	
	Optimized	110.455	78	11	A	
showProdotti.jsp (50 prodotti)	Baseline	634.085	85	18	A	
	Optimized	168.200	345	13	A	
index.jsp (50 prodotti)	Baseline	797.183	117	21	A	
	Optimized	284.108	107	19	A	
dashboard.jsp	Baseline	N/A	N/A	N/A	N/A	
	Optimized	283.924	107	18	A	

Table 4.1: Confronto delle metriche ambientali prima (Baseline) e dopo (Optimized) il refactoring.

- **modificaProdotto.jsp**: La riscrittura dello script `modificaProdotto.js` in puro Vanilla JS e l'ottimizzazione del rendering del form hanno ridotto il peso del 78,7% (da 100,0 KB a 21,3 KB), mantenendo un ottimo score EcoIndex pari a **95,0**.
- **paginaProdotto.jsp**: Il passaggio al JavaScript nativo e la semplificazione delle card delle azioni hanno ridotto il peso della pagina del 43,0% (da 196,7 KB a 112,2 KB) e ridotto le richieste da 12 a 10.
- **paginaUtente.jsp**: La rimozione di jQuery e dei tag di formattazione non semantici ha abbattuto il peso del 42,0% (da 190,8 KB a 110,7 KB) e ridotto le richieste HTTP da 13 a 10.

- **registrazione.jsp**: Seguendo lo stesso approccio di `login.jsp`, sono state eliminate le dipendenze CDN inutilizzate e consolidato il layout, riducendo il peso del 96,8% (da 104,9 KB a 3,4 KB), le richieste HTTP da 9 a 5, e portando lo score EcoIndex a **96,0**.
- **visualizzaOrdine.jsp**: La rimozione di script non utilizzati e l'appiattimento del DOM hanno ridotto il peso del 41,0% (da 186,5 KB a 110,0 KB) e le richieste HTTP da 13 a 11.
- **visualizzaOrdineAdmin.jsp**: Riscritto interamente in Vanilla JS, ha permesso di abbattere il peso del 80,2% (da 96,3 KB a 19,1 KB) e ridurre le richieste HTTP da 16 a 15, mantenendo il punteggio EcoIndex stabile a **95,0**.
- **wishlist.jsp**: La rimozione di jQuery e lo switch a miniature compresse con caricamento lazy hanno ridotto il peso del 40,7% (da 186,3 KB a 110,5 KB), i nodi DOM da 87 a 78 e le richieste HTTP da 12 a 11, aumentando il punteggio EcoIndex a **91,0**.
- **showProdotti.jsp**: Il passaggio all'utilizzo esclusivo delle miniature dei prodotti compresse in formato WebP ha ridotto drasticamente il peso della griglia da 634,1 KB a 168,2 KB (risparmio del 73,5%) e le richieste da 18 a 13, nonostante la pagina sia stata sottoposta a stress-test caricando contemporaneamente ben 50 prodotti.
- **index.jsp**: La ristrutturazione semantica del menu e l'ottimizzazione del carosello principale hanno abbattuto il peso della landing page da 797,2 KB a 284,1 KB (risparmio del 64,4%) e ridotto le richieste HTTP da 21 a 19, innalzando lo score EcoIndex a **88,0**.
- **dashboard.jsp**: Nella versione baseline l'assenza di sessione attiva causava il blocco/crash dell'analisi per reindirizzamento non gestito (N/A). Grazie all'introduzione del `SessionInjectionFilter` la pagina è stata scansionata con successo, registrando un peso di 283,9 KB, 107 nodi DOM, 18 richieste HTTP e un ottimo score EcoIndex pari a **88,0 (Grade A)**.

4.3.1 Quantificazione dell'Ottimizzazione Globale

L'efficacia complessiva del piano di refactoring e ottimizzazione ecologica del front-end di 2Chance può essere scientificamente quantificata aggregando e confrontando le metriche totali delle 15 pagine analizzabili in entrambi gli scenari:

1. **Riduzione del Payload di Rete (Byte Risparmiati)**: Il peso complessivo trasferito per il caricamento delle 15 pagine del portale è sceso da **3,93 MB** (3.932.064 byte) della versione Baseline a soli **1,49 MB** (1.491.605 byte) della versione Ottimizzata. Questo rappresenta una riduzione netta del **62,1%** dei dati trasmessi sulla rete globale, abbattendo drasticamente le emissioni di CO2 e il consumo di risorse energetiche nei router e switch di instradamento.
2. **Riduzione delle Richieste HTTP**: Il numero totale di richieste HTTP necessarie per caricare l'intero portale è sceso da **194** a **164**, registrando un risparmio netto del **15,5%**. Questo abbattimento riduce sensibilmente i tempi di round-trip time, l'overhead delle negoziazioni TCP/TLS e la congestione delle risorse di rete lato client e server.
3. **Efficienza del DOM sotto Carico**: Nelle griglie dinamiche (come `showProdotti.jsp` e `index.jsp`), nonostante il numero di articoli pre-caricati sia stato quadruplicato (da 12 a 50 articoli) per simulare un carico di stress reale, l'appiattimento strutturale del markup e l'introduzione del Lazy Loading hanno evitato la proliferazione incontrollata dei nodi DOM e il pre-caricamento di immagini non visibili, preservando la CPU e la memoria RAM del client.

4.3.2 Nota Metodologica sulle Fluttuazioni del Punteggio EcoIndex

Durante le fasi di audit con lo strumento GreenIT, si è osservata una leggera fluttuazione del punteggio globale EcoIndex (ad esempio da 89.01 a 87.37) a fronte di una risoluzione completa di tutte le inefficienze segnalate. Questa variazione è dovuta esclusivamente a un **effetto di caching asincrono** del browser di test:

- Nelle prime misurazioni (Stage 3), risorse pesanti come il file di font locale `LEMONMILK-Medium.otf` (93 KB) e librerie JavaScript CDN (jQuery, FontAwesome) risultavano pre-caching a livello del browser di sviluppo, riducendo fittiziamente il peso di rete a 37 KB e il numero di richieste a 17.
- Nelle misurazioni successive (Stage 4 e 5), a causa dell'introduzione del caricamento incrociato cross-origin (necessario per eliminare i cookie di sessione dagli asset statici), il browser ha dovuto rieseguire un caricamento completo (clean cache) di tutte le 21 risorse (inclusi i font, i fogli di stampa dedicati e le dipendenze CDN), portando la dimensione misurata a 211 KB.

Nonostante il punteggio matematico EcoIndex mostri una lieve flessione a causa del volume totale di asset attivamente dichiarati e scansionati a cache vuota, l'efficienza ecologica reale del sistema è salita al massimo livello: le risorse testuali sono compresse al 100%, gli asset statici sono interamente memorizzati nella cache locale dopo la prima richiesta (0 byte trasferiti successivamente), i cookie sono azzerati per le risorse pubbliche e non si verificano più costosi ridimensionamenti grafici lato client o errori di instradamento HTTP 404/500.

Chapter 5

Monitoraggio Licenze Open Source

5.1 Analisi della Compliance e dei Rischi Legali (FOSSA)

5.1.1 Sostenibilità Legale ed Economica: Software Composition Analysis e Conformità delle Licenze

Nel perimetro del Sustainable Software Engineering, la sostenibilità di un artefatto non è misurabile esclusivamente in termini di Watt consumati o emissioni di anidride carbonica; essa è intrinsecamente e profondamente legata alla vitalità economica a lungo termine del prodotto e alla mitigazione dei rischi legali. Un software che possiede un'architettura ambientalmente impeccabile, ma il cui codice viola sistematicamente le licenze intellettuali di terze parti, è destinato a un inevitabile fallimento economico a causa di cause legali, sanzioni e ingiunzioni di blocco della distribuzione [?].

L'ecosistema di sviluppo Java, sul quale si erge l'intera piattaforma 2Chance, si appoggia strutturalmente su repository immensi di dipendenze esterne (Maven Central). Ogni libreria dichiarata nel file `pom.xml` eredita a sua volta decine di librerie transitive, creando un albero di dipendenze labirintico e opaco [?]. Nel precedente ciclo di sviluppo focalizzato sulla Dependability, il team ha già dimostrato proattività in questo ambito utilizzando lo strumento Snyk per condurre una Software Composition Analysis (SCA) [?]. Questa scansione si era concentrata esclusivamente sull'individuazione e la risoluzione di vulnerabilità di sicurezza (CVE), forzando l'aggiornamento di componenti critici e obsoleti come il `mysql-connector` e le librerie `commons-io` [?]. Tuttavia, la sicurezza rappresenta unicamente un vettore del rischio associato alla supply chain del software.

5.1.2 Monitoraggio delle Licenze Open Source tramite FOSSA

Per colmare questa profonda lacuna e garantire una sostenibilità legale ferrea, il progetto 2Chance integrerà l'uso di FOSSA, una piattaforma di classe enterprise dedicata alla governance delle dipendenze e alla compliance delle licenze open source [?]. FOSSA non si limita a leggere i metadati manifesti nei file `.pom`, ma offre una tecnologia di scansione full-text (Audit-Grade License Scanning) capace di analizzare il codice sorgente grezzo delle dipendenze con una precisione dichiarata del 99.8% [?]. Questo meccanismo permette di rilevare intestazioni di copyright, identificare stralci di codice copiato in modo improprio ed estrarre i vincoli normativi anche da varianti di licenze alterate o non standard (es. licenze MIT con clausole di attribuzione modificate) [?].

L'integrazione di FOSSA all'interno del progetto 2Chance avverrà sfruttando il plugin Maven nativo della piattaforma (`fossa-maven-plugin`) o la CLI dedicata [?]. Data la complessità degli alberi delle dipendenze in Java, FOSSA eseguirà un'analisi profonda intercettando

il comando `mvn dependency:tree` o avvalendosi di versioni embedded del plugin per mappare accuratamente sia le dipendenze dirette (dichiarate esplicitamente dagli sviluppatori di 2Chance) sia quelle transitive (ereditate indirettamente) [?]. L'obiettivo strategico di questa imponente operazione di conformità è declinato su tre livelli:

1. **Identificazione Sistematica del Rischio Copyleft:** Il nucleo del problema legale risiede nella classificazione delle licenze in due macro-categorie: permissive (es. MIT, Apache 2.0) e copyleft (es. GPL v2/v3, AGPL) [?]. Le licenze permissive impongono vincoli minimi (principalmente la citazione), mentre le licenze copyleft sono “virali”: l'inclusione, anche accidentale, di una singola libreria GPL all'interno di un progetto proprietario closed-source può obbligare legalmente gli autori a rilasciare gratuitamente l'intero codice sorgente dell'applicazione, annullando ogni potenziale di monetizzazione e protezione del segreto industriale [?]. FOSSA mapperà l'intero albero delle dipendenze ed evidenzierà con alert critici eventuali violazioni di policy legate alla presenza non desiderata di licenze copyleft [?].
2. **Automazione Assoluta degli Obblighi di Attribuzione:** La stragrande maggioranza delle licenze open source (ivi compresa l'ubiqua Apache 2.0) impone obblighi legali stringenti per la distribuzione, tra cui la conservazione inalterata dei file di NOTA (NOTICE) e la generazione di documenti di attribuzione visibili all'utente finale [?]. FOSSA dispone di funzionalità di Automated Attributions, in grado di compilare e aggiornare dinamicamente e automaticamente l'elenco di tutte le licenze e i crediti richiesti, sollevando il team di sviluppo da una prassi manuale, tediosa e ad altissimo rischio di errore umano [?].
3. **Generazione della Software Bill of Materials (SBOM):** Una compliance proattiva richiede la trasparenza della catena di approvvigionamento [?]. FOSSA favorirà l'esportazione automatizzata di una SBOM (Distinta Base del Software) aderente agli standard industriali come SPDX o CycloneDX [?]. La fornitura di una SBOM chiara è un requisito sempre più pressante nei bandi di gara governativi (si veda l'executive order statunitense sulla cybersecurity) e costituisce un pilastro per l'ottenimento di certificazioni aziendali internazionali, come la conformità alle specifiche ISO/IEC 5230 dell'OpenChain Project [?].

La risoluzione dei conflitti legali identificati da FOSSA — che spesso si traduce nella rimozione di una libreria utilitaristica viziata da una licenza virale e nella sua sostituzione con un'alternativa funzionalmente equivalente ma dotata di licenza permissiva (MIT o Apache) — rappresenta un intervento che, pur non modificando le funzionalità visibili all'utente, consolida in modo formidabile l'integrità strutturale ed economica del sistema 2Chance, rendendolo pronto per un'ipotetica adozione su scala enterprise.

5.1.3 Esecuzione dell'Analisi FOSSA

L'analisi è stata eseguita mediante la CLI ufficiale di FOSSA (versione 3.17.8, revisione e21170d20446), installata in ambiente Windows tramite lo script di installazione ufficiale¹. La CLI è stata invocata in modalità `-output`, che consente l'estrazione locale del grafo delle dipendenze e la serializzazione dei risultati in formato JSON senza necessitare di un'autenticazione remota verso i server della piattaforma FOSSA.

¹<https://raw.githubusercontent.com/fossas/fossa-cli/master/install-latest.ps1>

Listing 5.1: Comandi per l'installazione e l'esecuzione della CLI FOSSA

```
# Installazione della CLI FOSSA (PowerShell)
Set-ExecutionPolicy Bypass -Scope Process -Force
iex ((New-Object System.Net.WebClient).DownloadString(
    'https://raw.githubusercontent.com/fossas/fossa-cli/
    master/install-latest.ps1'))

# Analisi delle dipendenze (output locale)
fossa.exe analyze --output

# Albero delle dipendenze Maven
mvn dependency:tree

# Download dei metadati delle licenze
mvn org.codehaus.mojo:license-maven-plugin:2.4.0:
    download-licenses
```

La CLI ha identificato automaticamente il progetto Maven dalla presenza del file `pom.xml` nella directory radice, rilevando il target `maven@.:\:groupId:2chance`. L'analisi è stata condotta mediante la strategia di parsing statico del file `pom.xml`, poiché la strategia dinamica basata sul `fossa-maven-plugin` embedded non è risultata disponibile nell'ambiente di esecuzione. La scansione ha restituito un esito positivo: **1 progetto analizzato, 1 riuscito, 0 falliti**, con 4 avvertimenti non bloccanti relativi alla strategia di analisi dinamica.

Per completare l'inventario delle licenze — informazione che la CLI FOSSA risolve integralmente solo tramite la piattaforma cloud — è stato impiegato il plugin Maven `license-maven-plugin` (versione 2.4.0), il quale ha scaricato e catalogato i metadati di licenza per tutte le 23 dipendenze (dirette e transitive), persistendo i risultati nel file `licenses.xml` e le copie integrali dei testi di licenza nella directory `src/fossa/licenses/`.

5.1.4 Albero delle Dipendenze del Progetto 2Chance

L'analisi combinata di FOSSA CLI e del comando `mvn dependency:tree` ha mappato l'intero albero delle dipendenze del progetto `groupId:2chance:war:1.0-SNAPSHOT`, identificando **15 dipendenze dirette** (dichiarate esplicitamente nel `pom.xml`) e **8 dipendenze transitive** (ereditate indirettamente), per un totale di **23 artefatti**. Lo scope di ciascuna dipendenza è fondamentale ai fini dell'analisi legale: le dipendenze con scope `compile` vengono incluse nel pacchetto WAR distribuito, quelle con scope `provided` sono fornite dal container a runtime (Tomcat) e non distribuite, mentre quelle con scope `test` sono utilizzate esclusivamente durante la fase di testing e non vengono mai consegnate all'utente finale.

Listing 5.2: Albero delle dipendenze Maven del progetto 2Chance

```
groupId:2chance:war:1.0-SNAPSHOT
+- javax.servlet:javax.servlet-api:4.0.1 [provided]
+- org.json:json:20231013 [compile]
+- commons-io:commons-io:2.14.0 [compile]
+- org.apache.tomcat:tomcat-jdbc:9.0.111 [compile]
| \- org.apache.tomcat:tomcat-juli:9.0.111 [compile]
+- com.mysql:mysql-connector-j:9.5.0 [compile]
| \- com.google.protobuf:protobuf-java:4.31.1 [compile]
+- org.apache.taglibs:taglibs-standard-spec:1.2.5 [compile]
+- org.apache.taglibs:taglibs-standard-impl:1.2.5 [compile]
+- org.junit.jupiter:junit-jupiter-api:5.9.1 [test]
| +- org.opentest4j:opentest4j:1.2.0 [test]
```

```

| +- org.junit.platform:junit-platform-commons:1.9.1 [test]
| \- org.apiguardian:apiguardian-api:1.1.2 [test]
+- org.junit.jupiter:junit-jupiter-engine:5.9.1 [test]
| \- org.junit.platform:junit-platform-engine:1.9.1 [test]
+- org.junit.jupiter:junit-jupiter-params:5.9.1 [test]
+- org.mockito:mockito-core:5.11.0 [test]
| +- net.bytebuddy:byte-buddy:1.14.12 [test]
| +- net.bytebuddy:byte-buddy-agent:1.14.12 [test]
| \- org.objenesis:objenesis:3.3 [test]
+- org.mockito:mockito-junit-jupiter:5.11.0 [test]
+- org.pitest:pitest-parent:1.1.10 [compile]
+- org.openjdk.jmh:jmh-core:1.37 [compile]
| +- net.sf.jopt-simple:jopt-simple:5.0.4 [compile]
| \- org.apache.commons:commons-math3:3.6.1 [compile]
\-- org.openjdk.jmh:jmh-generator-annprocess:1.37 [provided]

```

5.1.5 Inventario Completo delle Licenze

La Tabella 5.1 presenta l’inventario esaustivo delle licenze per ciascuna dipendenza del progetto 2Chance, classificando ogni artefatto secondo la categoria di rischio legale associata: **Permissiva** (rischio nullo), **Copyleft Debole** (rischio basso, generalmente limitato alle modifiche della libreria stessa) e **Copyleft con Eccezione** (rischio medio, mitigato da clausole specifiche che consentono il linking senza propagazione virale).

Table 5.1: Inventario delle licenze delle dipendenze del progetto 2Chance.

Dipendenza	Scope	Licenza	Categoria
commons-io:commons-io 2.14.0	compile	Apache-2.0	Permissiva
org.json:json 20231013	compile	Public Domain	Permissiva
org.apache.tomcat:tomcat-jdbc 9.0.111	compile	Apache-2.0	Permissiva
org.apache.tomcat:tomcat-juli 9.0.111	compile	Apache-2.0	Permissiva
com.google.protobuf:protobuf-java 4.31.1	compile	BSD-3-Clause	Permissiva
org.apache.taglibs:taglibs-standard-spec 1.2.5	compile	Apache-2.0	Permissiva
org.apache.taglibs:taglibs-standard-impl 1.2.5	compile	Apache-2.0	Permissiva
org.pitest:pitest-parent 1.1.10	compile	Apache-2.0	Permissiva
net.sf.jopt-simple:jopt-simple 5.0.4	compile	MIT	Permissiva
org.apache.commons:commons-math3 3.6.1	compile	Apache-2.0	Permissiva
com.mysql:mysql-connector-j 9.5.0	compile	GPLv2 + FOSS Exc.	Copyleft+Exc.
org.openjdk.jmh:jmh-core 1.37	compile	GPLv2 + CE	Copyleft+Exc.
javax.servlet:javax.servlet-api 4.0.1	provided	CDDL + GPLv2 + CE	Copyleft+Exc.

Continua nella pagina successiva

Dipendenza	Scope	Licenza	Categoria
org.openjdk.jmh:jmh-generator-asm 1.37	compile	GPLv2 + CE	Copyleft+Exc.
org.junit.jupiter:junit-jupiter-api 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.jupiter:junit-jupiter-engine 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.jupiter:junit-jupiter-params 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.platform:junit-platform-commons 1.9.1	test	EPL-2.0	Copyleft Debole
org.junit.platform:junit-platform-engine 1.9.1	test	EPL-2.0	Copyleft Debole
org.mockito:mockito-core 5.11.0	test	MIT	Permissiva
org.mockito:mockito-junit-jupiter 5.11.0	test	MIT	Permissiva
net.bytebuddy:byte-buddy 1.14.12	test	Apache-2.0	Permissiva
net.bytebuddy:byte-buddy-agent 1.14.12	test	Apache-2.0	Permissiva
org.objenesis:objenesis 3.3	test	Apache-2.0	Permissiva
org.opentest4j:opentest4j 1.2.0	test	Apache-2.0	Permissiva
org.apiguardian:apiguardian-api 1.1.2	test	Apache-2.0	Permissiva

5.1.6 Distribuzione delle Licenze

L'analisi rivela una distribuzione favorevole delle licenze nell'ecosistema delle dipendenze di 2Chance. La Tabella 5.2 riassume la distribuzione per tipologia di licenza, evidenziando una netta prevalenza di licenze permissive.

Il dato più rilevante è l'assenza totale di licenze copyleft pure (GPL o AGPL senza eccezioni): **nessuna violazione critica è stata rilevata**. Tutte le licenze appartenenti alla famiglia GPL presenti nel progetto includono eccezioni esplicite (Universal FOSS Exception o Classpath Exception) che permettono il linking con codice proprietario o FOSS senza innescare l'effetto virale caratteristico della GPL [?].

5.1.7 Valutazione del Rischio Copyleft

L'analisi FOSSA ha identificato quattro dipendenze che richiedono un'attenzione specifica a causa della natura copyleft delle loro licenze. Di seguito si presenta la valutazione dettagliata per ciascuna di esse.

mysql-connector-j (GPLv2 + Universal FOSS Exception v1.0) — Rischio Medio

Il connettore MySQL per Java (com.mysql:mysql-connector-j:9.5.0), dichiarato con scope **compile** e quindi incluso nel pacchetto WAR distribuito, è rilasciato sotto licenza **GNU General Public License v2** con la **Universal FOSS Exception v1.0**. La GPLv2 è una licenza copyleft forte che, in assenza di eccezioni, imporrebbe il rilascio dell'intero codice sorgente

Tipologia di Licenza	Conteggio	Percentuale
Apache-2.0	13	50,0%
MIT	3	11,5%
BSD-3-Clause	1	3,8%
Public Domain	1	3,8%
Totale Permissive	18	69,2%
Eclipse Public License 2.0 (EPL-2.0)	5	19,2%
GPLv2 + Universal FOSS Exception	1	3,8%
GPLv2 + Classpath Exception	2	7,7%
Totale Copyleft/Copyleft con Eccezione	8	30,8%
GPL/AGPL pura (senza eccezioni)	0	0,0%

Table 5.2: Distribuzione delle tipologie di licenza nell'albero delle dipendenze di 2Chance.

dell'applicazione collegata. Tuttavia, la Universal FOSS Exception concede esplicitamente il permesso di linkare il connettore con qualsiasi software rilasciato sotto una licenza riconosciuta dalla Free Software Foundation o dall'Open Source Initiative, senza attivare gli obblighi copyleft della GPL [?].

Poiché il progetto 2Chance utilizza esclusivamente dipendenze con licenze FOSS riconosciute (Apache-2.0, MIT, BSD-3-Clause, EPL-2.0), l'eccezione si applica integralmente e il rischio legale è **mitigato**. Tuttavia, qualora il progetto venisse in futuro distribuito come software proprietario closed-source, questa dipendenza dovrebbe essere sostituita con un'alternativa compatibile, come il MariaDB Connector/J rilasciato sotto licenza LGPL-2.1.

jmh-core (GPLv2 + Classpath Exception) — Rischio Medio

La libreria Java Microbenchmark Harness (`org.openjdk.jmh:jmh-core:1.37`), attualmente dichiarata con scope `compile`, è rilasciata sotto **GPLv2 con Classpath Exception**. La Classpath Exception è una clausola standard adottata dall'ecosistema OpenJDK che permette di creare applicazioni che utilizzano le librerie senza essere soggette agli obblighi di distribuzione del codice sorgente imposti dalla GPL [?].

Il rischio principale risiede nel fatto che JMH è uno strumento di benchmarking, concepito esclusivamente per la profilazione delle prestazioni durante lo sviluppo, e **non dovrebbe essere incluso nel pacchetto WAR di produzione**. La sua presenza con scope `compile` rappresenta un'anomalia architetturale: la modifica dello scope a `test` nel file `pom.xml` eliminerebbe completamente qualsiasi rischio di distribuzione, poiché le dipendenze con scope `test` non vengono mai incluse nell'artefatto finale.

jmh-generator-annprocess (GPLv2 + Classpath Exception) — Rischio Basso

Il processore di annotazioni di JMH (`org.openjdk.jmh:jmh-generator-annprocess:1.37`) condivide la stessa licenza GPLv2 + Classpath Exception del core di JMH. Tuttavia, essendo già correttamente dichiarato con scope `provided`, non viene incluso nel pacchetto WAR distribuito. Il rischio legale è pertanto **nullo** per questa dipendenza.

javax.servlet-api (CDDL + GPLv2 con Classpath Exception) — Rischio Nullo

La Servlet API (`javax.servlet:javax.servlet-api:4.0.1`) è rilasciata sotto doppia licenza: **CDDL** (Common Development and Distribution License, una licenza permissiva) e **GPLv2 con Classpath Exception**. Essendo dichiarata con scope `provided`, viene fornita dal server Tomcat a runtime e non è inclusa nel pacchetto WAR. Il rischio legale è pertanto **nullo**.

5.1.8 Sintesi del Rischio e Raccomandazioni

La Tabella 5.3 riassume la valutazione complessiva del rischio legale per il progetto 2Chance.

Livello di Rischio	N.	Dettaglio
Critico (GPL/AGPL pura)	0	Nessuna violazione copyleft pura rilevata
Medio (Copyleft + Eccezione)	2	<code>mysql-connector-j</code> , <code>jmh-core</code>
Basso (Scope provided/test)	2	<code>jmh-generator-annprocess</code> , <code>javax.servlet-api</code>
Nessuno (Permissiva/Test)	22	Tutte le rimanenti dipendenze

Table 5.3: Sintesi della valutazione del rischio copyleft per il progetto 2Chance.

Sulla base dei risultati dell'analisi FOSSA e della catalogazione delle licenze, si formulano le seguenti raccomandazioni operative:

1. **Modifica dello scope di `jmh-core` da `compile` a `test`:** JMH è uno strumento di benchmarking e non ha alcuna utilità funzionale nell'applicazione in produzione. La sua inclusione nel WAR distribuito introduce un rischio legale evitabile e un peso inutile sull'artefatto finale. La modifica dello scope nel `pom.xml` è un intervento triviale che elimina completamente il rischio associato alla licenza GPLv2 + CE.
2. **Monitoraggio continuo di `mysql-connector-j`:** La Universal FOSS Exception garantisce la compatibilità attuale con l'ecosistema di licenze del progetto. Tuttavia, qualora 2Chance evolvesse verso un modello di distribuzione commerciale proprietario, il connettore MySQL dovrebbe essere sostituito con il MariaDB Connector/J (LGPL-2.1), funzionalmente equivalente e dotato di una licenza più permissiva per l'uso commerciale.
3. **Generazione del file `THIRD-PARTY-NOTICES`:** I testi integrali delle 8 licenze uniche sono stati scaricati nella directory `src/fossa/licenses/`. Si raccomanda la compilazione di un documento di attribuzione formale (`THIRD-PARTY-NOTICES.txt`) da includere nella distribuzione, al fine di soddisfare gli obblighi di attribuzione imposti dalle licenze Apache-2.0, MIT e BSD-3-Clause [?].
4. **Integrazione di FOSSA nel pipeline CI/CD:** Per garantire un monitoraggio proattivo e continuo, si raccomanda la registrazione di un account sulla piattaforma FOSSA (<https://app.fossa.com>) e la configurazione di una API key nel workflow GitHub Actions del progetto. Ciò abiliterebbe la scansione automatizzata ad ogni commit, la generazione di SBOM in formato SPDX/CycloneDX [?] e il blocco automatico delle pull request che introducono dipendenze con licenze non conformi alla policy del progetto.

5.1.9 Artefatti dell'Analisi

Tutti i risultati grezzi dell'analisi FOSSA e della catalogazione delle licenze sono stati persistiti nella directory `src/fossa/` del repository del progetto, garantendo la piena riproducibilità dell'analisi e la disponibilità dei dati per verifiche future. I file prodotti sono:

- `fossa-analyze-output.json`: output JSON grezzo della CLI FOSSA contenente il grafo completo delle dipendenze.
- `fossa-scan-summary.txt`: riepilogo testuale della scansione con avvertimenti diagnostici.
- `dependency-tree.txt`: output integrale del comando `mvn dependency:tree`.
- `licenses.xml`: metadati strutturati delle licenze per tutte le 23 dipendenze, generati dal `license-maven-plugin`.
- `licenses/`: directory contenente i testi integrali delle 8 licenze uniche scaricate automaticamente.

5.1.10 Interventi Correttivi Applicati

A seguito dell'analisi FOSSA, sono stati applicati due interventi correttivi al file `pom.xml` che migliorano significativamente il profilo di rischio legale del progetto senza alterare in alcun modo le funzionalità dell'applicazione. Entrambi gli interventi consistono nella modifica dello scope Maven di dipendenze utilizzate esclusivamente durante le fasi di sviluppo e testing.

Modifica dello Scope di `jmh-core` (`compile` → `test`)

La libreria Java Microbenchmark Harness (`org.openjdk.jmh:jmh-core:1.37`), rilasciata sotto licenza **GPLv2 con Classpath Exception**, era erroneamente dichiarata con scope `compile`, causandone l'inclusione nel pacchetto WAR distribuito. Un'analisi esaustiva del codice sorgente ha confermato che le importazioni di JMH (`import org.openjdk.jmh.annotations.*` e `import org.openjdk.jmh.infra.Blackhole`) sono presenti **esclusivamente** nei file di benchmark situati nella directory `src/jmh/java/` e in nessun file della directory principale `src/main/java/`. La modifica dello scope a `test` elimina la libreria e le sue dipendenze transitive (`jopt-simple`, `commons-math3`) dal WAR distribuito, rimuovendo completamente la licenza GPLv2 + CE dall'artefatto finale.

Listing 5.3: Modifica dello scope di `jmh-core` nel `pom.xml`

```
<!-- PRIMA (scope implicito = compile) -->
<dependency>
  <groupId>org.openjdk.jmh</groupId>
  <artifactId>jmh-core</artifactId>
  <version>${jmh.version}</version>
</dependency>

<!-- DOPO (scope esplicito = test) -->
<dependency>
  <groupId>org.openjdk.jmh</groupId>
  <artifactId>jmh-core</artifactId>
  <version>${jmh.version}</version>
  <scope>test</scope>
</dependency>
```

Modifica dello Scope di `pitest-parent` (`compile` → `test`)

La dipendenza `org.pitest:pitest-parent:1.1.10`, dichiarata come dipendenza POM senza scope esplicito (defaultando a `compile`), è utilizzata unicamente come supporto al plugin di mutation testing `pitest-maven`, già correttamente configurato nel profilo Maven `pitest`. Un'analisi del codice sorgente ha confermato che **nessun file Java** del progetto importa classi dal namespace `org.pitest`. La modifica dello scope a `test` è pertanto priva di rischi funzionali.

5.1.11 Impatto degli Interventi Correttivi

La Tabella 5.4 confronta il profilo di rischio legale del progetto 2Chance prima e dopo gli interventi correttivi applicati.

Livello di Rischio	Prima	Dopo
Critico (GPL/AGPL pura)	0	0
Medio (Copyleft + Eccezione, nel WAR)	2	1
Basso (Scope provided/test)	2	4
Nessuno (Permissiva / Non distribuito)	22	21

Table 5.4: Confronto del profilo di rischio legale prima e dopo gli interventi correttivi.

L'intervento più significativo riguarda la rimozione di `jmh-core` dallo scope `compile`: questa singola modifica ha eliminato la libreria GPLv2 + CE e le sue due dipendenze transitive (`jopt-simple` e `commons-math3`) dal pacchetto WAR distribuito. L'unica dipendenza copyleft che rimane nello scope `compile` è il connettore MySQL (`mysql-connector-j`), la cui Universal FOSS Exception garantisce la piena compatibilità con l'ecosistema di licenze FOSS del progetto.

Il risultato complessivo è una riduzione del 50% delle dipendenze a rischio medio distribuite nel WAR (da 2 a 1), senza alcuna alterazione delle funzionalità del sistema. Il progetto compila correttamente e tutti i test esistenti rimangono operativi.

5.1.12 Automazione degli Obblighi di Attribuzione

Per soddisfare pienamente il secondo obiettivo di conformità legale (Automazione Assoluta degli Obblighi di Attribuzione), si è provveduto a configurare e lanciare il plugin Maven dedicato alla gestione delle licenze per generare i documenti richiesti per la distribuzione del software.

Sono stati prodotti due artefatti chiave all'interno della directory `src/fossa/`:

- **THIRD-PARTY-NOTICES.txt**: File di testo contenente l'elenco completo di tutte le dipendenze di terze parti, con la rispettiva classificazione della licenza e il testo integrale del copyright e dei termini di licenza per ciascuna di esse. Questo file è pronto per essere incluso nella directory di distribuzione dell'applicazione per adempiere agli obblighi legali di attribuzione delle licenze permissive (quali Apache 2.0, MIT, BSD).
- **third-party-report.html**: Un report visuale in formato HTML che offre un'interfaccia interattiva per navigare l'inventario delle dipendenze e i relativi dettagli legali.

L'adozione di questa pipeline automatizzata garantisce che ad ogni build di rilascio, la documentazione sulle licenze venga rigenerata coerentemente con le dipendenze effettivamente introdotte o aggiornate nel `pom.xml`, sollevando il team di sviluppo da controlli manuali inclini all'errore umano e garantendo la conformità legale al 100% in modo continuo.

5.1.13 Generazione della Software Bill of Materials (SBOM)

Al fine di garantire la trasparenza della catena di approvvigionamento del software (*software supply chain*) ed adempiere alle moderne richieste di conformità legale (quali l'ISO/IEC 5230 dell'OpenChain Project o i requisiti in materia di cybersecurity per i bandi governativi), è stata automatizzata la generazione della **Software Bill of Materials (SBOM)** per l'applicazione 2Chance.

L'operazione è stata implementata integrando e lanciando il plugin Maven ufficiale per lo standard **CycloneDX** (uno standard industriale leader per la specifica di SBOM). Sono stati generati con successo due formati dello stesso documento all'interno della directory `src/fossa/`:

- `sbom.json`: La distinta base completa in formato JSON (CycloneDX v1.4), strutturata per essere analizzata in automatico da strumenti di vulnerability management o Dependency-Track.
- `sbom.xml`: Lo stesso report in formato XML, per la massima compatibilità e integrazione con altri parser aziendali standard.

Questi file descrivono in modo univoco e strutturato ciascun componente software incluso nel build, specificando per ognuno di essi: identificativi univoci (Package URL - *purl*), versione precisa, hash SHA-1/SHA-256 del binario e la classificazione delle licenze applicate. Questo consente ai clienti e agli auditor di effettuare controlli rapidi sulla sicurezza e sulla conformità delle licenze distribuite.